

LAPORAN TUGAS 7

SISTEM OPERASI



Dosen Pengampu:

Triawan Adi Cahyanto, M.Kom

Disusun Oleh:

Yoanre Louis A.H.P (2410651039)

PRODI TEKNIK INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH JEMBER

2025

DAFTAR ISI

BAB 1	4
PENDAHULUAN	4
1.1 LATAR BELAKANG	4
1.2 TUJUAN PRAKTIKUM	4
1.3 SPESIFIKASI SISTEM.....	4
BAB II	6
DASAR TEORI.....	6
2.1 MEMORY MANAGEMENT.....	6
2.2 CONTIGUOUS ALLOCATION.....	6
2.3 PAGING.....	6
2.4 PAGE TABLE STRUCTURES.....	6
2.5 SWAPPING	7
2.6 ARCHITECTURE	7
BAB III.....	8
JAWABAN TUGAS PRAKTIKUM	8
3.1 TUGAS 1.....	8
3.1.1 Deskripsi Tugas	8
3.1.2 Source Code	8
3.1.3 Screenshot Output.....	11
3.1.4 Penjelasan Implementasi.....	11
3.2 TUGAS 2.....	12
3.2.1 Deskripsi Tugas	12
3.2.2 Source Code	12
3.2.3 Screenshot Output.....	19
3.2.4 Penjelasan Implementasi.....	19
3.3 TUGAS 3.....	20
3.3.1 Deskripsi Tugas	20
3.3.2 Source Code	21
3.3.3 Screenshot Output.....	40
3.3.4 Penjelasan Implementasi.....	43
3.4 TUGAS 4.....	44

3.4.1 Deskripsi Tugas	44
3.4.2 Source Code	44
3.4.3 Screenshot Output.....	50
3.4.4 Penjelasan Implementasi.....	50
3.5 TUGAS 5.....	52
3.5.1 Deskripsi Tugas	52
3.5.2 Source Code	52
3.5.3 Screenshot Output.....	58
3.5.4 Penjelasan Implementasi.....	58
3.6 TUGAS 6.....	59
3.6.1 Deskripsi Tugas	59
3.6.2 Source Code	59
3.6.3 Screenshot Output.....	60
3.6.4 Penjelasan Implementasi.....	61
BAB IV	62
ANALISIS DAN PEMBAHASAN.....	62
4.1 ANALISIS TUGAS 1	62
4.2 ANALISIS TUGAS 2	65
4.3 ANALISIS TUGAS 3	66
4.4 ANALISIS TUGAS 4	68
4.5 ANALISIS TUGAS 5	70
4.6 ANALISIS TUGAS 6	71
BAB V	74
KESIMPULAN DAN SARAN	74
5.1 KESIMPULAN	74
5.2 SARAN.....	74
DAFTAR PUSTAKA	75

BAB 1

PENDAHULUAN

1.1 LATAR BELAKANG

Manajemen memori merupakan salah satu fungsi kritis dalam sistem operasi yang bertanggung jawab atas pengalokasian, pengelolaan, dan optimasi penggunaan memori utama oleh berbagai proses. Dalam sistem komputer modern, di mana banyak aplikasi berjalan secara bersamaan, efisiensi pengelolaan memori sangat menentukan kinerja dan stabilitas sistem. Teknik-teknik seperti paging, swapping, dan contiguous allocation dikembangkan untuk mengatasi masalah fragmentasi, mengoptimalkan penggunaan ruang memori, serta memungkinkan sistem menjalankan proses yang lebih besar daripada kapasitas memori fisik yang tersedia. Pemahaman mendalam tentang bagaimana sistem operasi mengorganisir memori—mulai dari layout segmen memori, algoritma alokasi, hingga mekanisme penggantian halaman—menjadi dasar bagi pengembangan perangkat lunak yang efisien dan andal. Praktikum ini dirancang untuk memberikan pengalaman langsung dalam mengimplementasikan dan menganalisis berbagai aspek manajemen memori melalui simulasi dan eksperimen berbasis kode.

1.2 TUJUAN PRAKTIKUM

1. Memahami struktur dan layout memori sebuah proses (text, data, heap, stack) serta menganalisis pengaruh ASLR (Address Space Layout Randomization) terhadap alokasi alamat.
2. Mengimplementasikan dan membandingkan performa berbagai algoritma alokasi memori kontigu (First-Fit, Best-Fit, Worst-Fit, Next-Fit) dalam mengelola fragmentasi eksternal.
3. Mensimulasikan sistem paging dengan algoritma penggantian halaman (FIFO dan LRU), menganalisis fenomena Belady's Anomaly, serta menguji berbagai struktur page table dan mekanisme swapping.

1.3 SPESIFIKASI SISTEM

1. Sistem Operasi: Ubuntu atau distribusi Linux lainnya, digunakan untuk menjalankan simulasi dan perintah analisis sistem seperti `lscpu`, `getconf`, dan `/proc/[pid]/maps`.

2. Bahasa Pemrograman: C, dengan compiler GCC, untuk mengimplementasikan seluruh program simulasi manajemen memori.
3. Perangkat Keras: Komputer dengan arsitektur x86-64 yang mendukung fitur manajemen memori modern seperti paging, TLB, dan hierarki cache.

BAB II

DASAR TEORI

2.1 MEMORY MANAGEMENT

Memory management merupakan fungsi utama sistem operasi yang bertugas mengatur penggunaan memori utama agar dapat digunakan secara efisien oleh berbagai proses. Sistem operasi bertanggung jawab dalam melakukan alokasi dan dealokasi memori, menjaga isolasi antar proses, serta memetakan alamat virtual ke alamat fisik. Dengan adanya manajemen memori, setiap proses seolah-olah memiliki ruang memori sendiri, meskipun secara fisik memori digunakan bersama. Teknik seperti virtual memory, paging, dan swapping digunakan untuk meningkatkan pemanfaatan memori, mengurangi fragmentasi, serta memastikan sistem dapat menjalankan banyak proses secara bersamaan dengan aman dan stabil.

2.2 CONTIGUOUS ALLOCATION

Contiguous allocation adalah metode pengalokasian memori di mana suatu proses ditempatkan pada satu blok memori yang berurutan. Metode ini sederhana dan mudah diimplementasikan, namun memiliki kelemahan berupa fragmentasi eksternal. Untuk mengatasi hal tersebut, digunakan beberapa algoritma seperti First-Fit, Best-Fit, dan Worst-Fit. First-Fit memilih blok kosong pertama yang mencukupi, Best-Fit memilih blok terkecil yang masih cukup, sedangkan Worst-Fit memilih blok terbesar. Pemilihan algoritma sangat memengaruhi efisiensi penggunaan memori dan kinerja sistem secara keseluruhan.

2.3 PAGING

Paging merupakan teknik manajemen memori yang membagi memori menjadi unit berukuran tetap yang disebut page dan frame. Dengan paging, proses tidak perlu menempati memori secara kontigu sehingga fragmentasi eksternal dapat dihindari. Alamat virtual dibagi menjadi page number dan offset, yang kemudian diterjemahkan menjadi alamat fisik menggunakan page table. Jika halaman yang dibutuhkan belum berada di memori utama, maka akan terjadi page fault dan sistem operasi akan memuat halaman tersebut dari penyimpanan sekunder. Paging memberikan fleksibilitas tinggi dalam pengelolaan memori modern.

2.4 PAGE TABLE STRUCTURES

Page table structures digunakan untuk mengelola pemetaan antara alamat virtual dan alamat fisik secara efisien. Pada sistem dengan ruang alamat besar, penggunaan single-level

page table dapat menimbulkan overhead memori yang besar. Oleh karena itu, dikembangkan struktur page table bertingkat seperti two-level page table. Struktur ini membagi page number menjadi beberapa level sehingga page table hanya dialokasikan saat dibutuhkan. Dengan pendekatan ini, penggunaan memori menjadi lebih efisien tanpa mengorbankan kemampuan translasi alamat, terutama pada sistem dengan virtual address space yang luas.

2.5 SWAPPING

Swapping adalah teknik manajemen memori yang memindahkan proses sementara dari memori utama ke penyimpanan sekunder, seperti hard disk, untuk mengosongkan ruang memori. Ketika proses tersebut kembali dibutuhkan, sistem operasi akan memuatnya kembali ke memori utama. Teknik ini memungkinkan sistem menjalankan lebih banyak proses dibandingkan kapasitas memori fisik yang tersedia. Namun, swapping memiliki kelemahan berupa overhead waktu karena akses disk lebih lambat dibandingkan memori utama. Oleh karena itu, swapping harus dikelola dengan strategi yang tepat agar tidak menurunkan kinerja sistem.

2.6 ARCHITECTURE

Arsitektur memori menggambarkan susunan hierarki memori dalam sistem komputer, yang terdiri dari cache, RAM, dan penyimpanan sekunder. Setiap lapisan memiliki karakteristik berbeda dalam hal kecepatan, kapasitas, dan biaya. Cache memiliki kecepatan akses paling tinggi namun kapasitas kecil, RAM berfungsi sebagai memori utama, sedangkan disk menyediakan kapasitas besar dengan kecepatan lebih rendah. Hierarki ini dirancang untuk mengoptimalkan kinerja sistem dengan menempatkan data yang sering diakses pada memori yang lebih cepat, sehingga waktu akses keseluruhan dapat diminimalkan.

BAB III

JAWABAN TUGAS PRAKTIKUM

3.1 TUGAS 1

3.1.1 Deskripsi Tugas

Tugas 1 bertujuan untuk menganalisis memory layout suatu proses secara lebih mendalam melalui modifikasi program `address_demo.c`. Pada tugas ini, mahasiswa diminta menambahkan fitur alokasi array statis dan dinamis untuk mengamati perbedaan alamat memori, membuat beberapa fungsi guna membandingkan alamat pada text segment, serta mengimplementasikan fungsi rekursif untuk melihat perubahan alamat stack pada setiap level pemanggilan. Selain itu, alamat string literals juga diamati. Hasil eksekusi program kemudian dianalisis dengan menghitung jarak antarsegmen memori, membandingkan perubahan alamat pada beberapa kali eksekusi, serta mencocokkannya dengan informasi memory map pada `/proc/[pid]/maps` untuk memahami pengaruh ASLR dan hak akses setiap segmen memori.

3.1.2 Source Code

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

/* prototype */
int main();

/* variabel global */
int global_var = 100;
int static_array[3] = {10, 20, 30};
const char *text = "Main Memory Praktikum";

/* fungsi biasa */
void func1() {}
void func2() {}
void func3() {}
```

```

/* fungsi rekursif */
void recursive(int level) {
    int x = level;
    printf("  Level %d local_var address: %p value: %d\n",
        level, (void*)&x, x);

    if (level < 2)
        recursive(level + 1);
}

void print_memory() {
    int local = 5;
    int *heap1 = malloc(sizeof(int));
    int *heap2 = malloc(sizeof(int));
    int *dyn_array = malloc(3 * sizeof(int));

    /* isi nilai */
    *heap1 = 50;
    *heap2 = 75;
    dyn_array[0] = 1;

    printf("\n=== ENHANCED MEMORY LAYOUT ===\n\n");

    printf("1. Text Segment:\n");
    printf("  main() : %p\n", (void*)main);
    printf("  func1() : %p\n", (void*)func1);
    printf("  func2() : %p\n", (void*)func2);
    printf("  func3() : %p\n\n", (void*)func3);

    printf("2. Data Segment:\n");
    printf("  global_var address: %p value: %d\n",
        (void*)&global_var, global_var);
    printf("  static_array[0] address: %p value: %d\n",

```

```

        (void*)&static_array[0], static_array[0]);
printf("  string literal address: %p value: %s\n\n",
        (void*)text, text);

printf("3. Heap:\n");
printf("  heap1 address: %p value: %d\n",
        (void*)heap1, *heap1);
printf("  heap2 address: %p value: %d\n",
        (void*)heap2, *heap2);
printf("  dyn_array[0] address: %p value: %d\n\n",
        (void*)&dyn_array[0], dyn_array[0]);

printf("4. Stack:\n");
printf("  local address: %p value: %d\n\n",
        (void*)&local, local);

printf("4. Stack (Recursive Calls):\n");
recursive(0);

free(heap1);
free(heap2);
free(dyn_array);
}

int main() {
    printf("PID: %d\n", getpid());
    print_memory();

    printf("\nTekan Enter untuk melihat memory map...");
    getchar();

    char cmd[100];
    sprintf(cmd, "cat /proc/%d/maps", getpid());

```

```
system(cmd);
```

```
return 0;
```

```
}
```

3.1.3 Screenshot Output

```
dany@LAPTOP-GR3L2V1D:~/praktikum-memory/background$ ./address_demo
PID: 675

=== ENHANCED MEMORY LAYOUT ===

1. Text Segment:
  main() : 0x61d1c1f654b
  func1() : 0x61d1c1f6249
  func2() : 0x61d1c1f6254
  func3() : 0x61d1c1f625f

2. Data Segment:
  global_var address: 0x61d1c1f9010 value: 100
  static_array[0] address: 0x61d1c1f9018 value: 10
  string literal address: 0x61d1c1f9008 value: Main Memory Praktikum

3. Heap:
  heap1 address: 0x61d1f13fe6b0 value: 50
  heap2 address: 0x61d1f13fe6d0 value: 75
  dyn_array[0] address: 0x61d1f13fe6f0 value: 1

4. Stack:
  local address: 0x7ffd0f6d0bc value: 5

4. Stack (Recursive Calls):
  Level 0 local_var address: 0x7ffd0f6d0bc value: 0
  Level 1 local_var address: 0x7ffd0f6d0cd value: 1
  Level 2 local_var address: 0x7ffd0f6d0c34 value: 2

Tekan Enter untuk melihat memory map...
61d1c1f5000-61d1c1f6000 r--p 00000000 08:30 43507 /home/dany/praktikum-memory/background/address_demo
61d1c1f6000-61d1c1f7000 r-xp 00001000 08:30 43507 /home/dany/praktikum-memory/background/address_demo
61d1c1f7000-61d1c1f8000 r--p 00002000 08:30 43507 /home/dany/praktikum-memory/background/address_demo
61d1c1f8000-61d1c1f9000 r--p 00002000 08:30 43507 /home/dany/praktikum-memory/background/address_demo
61d1c1f9000-61d1c1fa000 rw-p 00003000 08:30 43507 /home/dany/praktikum-memory/background/address_demo
61d1f13fe000-61d1f140000 rw-p 00000000 00:00 0 [heap]
76127cc28000-76127cc28000 r--p 00000000 00:00 0 /usr/lib/x86_64-linux-gnu/libc.so.6
76127cc28000-76127cc28000 r-xp 00028000 08:30 39744 /usr/lib/x86_64-linux-gnu/libc.so.6
76127cc28000-76127cc28000 r--p 00028000 08:30 39744 /usr/lib/x86_64-linux-gnu/libc.so.6
76127cc28000-76127cc28000 r-xp 001b0000 08:30 39744 /usr/lib/x86_64-linux-gnu/libc.so.6
76127cc28000-76127cc28000 r--p 001fe000 08:30 39744 /usr/lib/x86_64-linux-gnu/libc.so.6
76127cc28000-76127cc28000 r-xp 00202000 08:30 39744 /usr/lib/x86_64-linux-gnu/libc.so.6
76127cc28000-76127cc28000 r--p 00000000 00:00 0 [vdso]
76127cc28000-76127cc28000 r-xp 00000000 00:00 0 [vdso]
76127cee7000-76127cee7000 r--p 00000000 00:00 0 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
76127cee7000-76127cee7000 r-xp 00000000 00:00 0 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
76127cf1e000-76127cf1e000 r--p 00001000 08:30 39741 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
76127cf1e000-76127cf1e000 r-xp 0002c000 08:30 39741 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
76127cf28000-76127cf28000 r--p 00036000 08:30 39741 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
76127cf28000-76127cf28000 r-xp 00038000 08:30 39741 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
76127cf28000-76127cf28000 r--p 00000000 00:00 0 [stack]
76127cf28000-76127cf28000 r-xp 00000000 00:00 0 [vdso]
76127cf28000-76127cf28000 r--p 00000000 00:00 0 [vdso]
```

3.1.4 Penjelasan Implementasi

- Preprocessing - Header files (stdio.h, stdlib.h, unistd.h) dimasukkan, prototype fungsi didefinisikan
- Compilation - Kode C dikompilasi ke assembly termasuk fungsi rekursif recursive() dan multiple pointer
- Assembly - Assembly code diubah menjadi object file berisi machine code
- Linking - Object file digabung dengan library C menghasilkan executable
- Loading - OS memuat executable ke memori:
 - Text Segment: Kode fungsi main(), print_memory(), recursive(), func1(), func2(), func3()
 - Data Segment: Variabel global_var = 100, array static_array, string literal "Main Memory Praktikum"
- Stack Setup - Frame stack untuk main() dibuat

- Main Execution - main() mulai dieksekusi, menampilkan PID proses
- Function Call - print_memory() dipanggil, frame stack baru dibuat
- Heap Allocation - Tiga kali malloc() dipanggil untuk alokasi heap memory
- Memory Layout Display - Program mencetak semua alamat:
 - Fungsi di text segment (main, func1-3)
 - Variabel global di data segment
 - Alokasi dinamis di heap
 - Variabel lokal di stack
 - Variabel rekursif menunjukkan frame stack berbeda
- Recursive Demonstration - recursive() dipanggil secara rekursif, menunjukkan alamat stack yang berbeda untuk setiap level
- Memory Release - free() melepas semua memori heap
- Memory Mapping - Program menampilkan /proc/[PID]/maps untuk melihat layout memori aktual
- Termination - Program selesai, semua resource dibersihkan

3.2 TUGAS 2

3.2.1 Deskripsi Tugas

Tugas 2 bertujuan untuk mengembangkan program memory allocator dengan menambahkan algoritma Next-Fit serta fitur analisis fragmentasi dan performa. Mahasiswa diminta mengimplementasikan Next-Fit sebagai variasi dari First-Fit, menghitung tingkat fragmentasi memori, dan melakukan benchmark terhadap empat algoritma alokasi memori. Hasil pengujian dianalisis melalui tabel perbandingan waktu eksekusi, fragmentasi, dan kegagalan alokasi. Selain itu, mahasiswa diminta menyusun grafik visualisasi alokasi memori untuk memahami perbedaan perilaku tiap algoritma. Tugas ini menekankan kemampuan analisis efisiensi dan trade-off dalam manajemen memori.

3.2.2 Source Code

```
#include <stdio.h>
#include <time.h>

#define MAX_BLOCKS 100
```

```

typedef struct {
    int size;
    int free;    // 1 = free, 0 = allocated
    int process_id;
} Block;

Block memory[MAX_BLOCKS];
int block_count = 1;
int last_alloc_index = 0;

/* ===== INIT MEMORY ===== */
void init_memory(int total_size) {
    memory[0].size = total_size;
    memory[0].free = 1;
    memory[0].process_id = -1;
    block_count = 1;
    last_alloc_index = 0;
}

/* ===== DISPLAY ===== */
void display_memory() {
    printf("\nMemory State:\n");
    for (int i = 0; i < block_count; i++) {
        if (memory[i].free)
            printf("Block %d: FREE | Size = %d\n", i, memory[i].size);
        else
            printf("Block %d: P%d | Size = %d\n", i, memory[i].process_id, memory[i].size);
    }
}

/* ===== DEALLOCATE ===== */
void deallocate(int pid) {
    for (int i = 0; i < block_count; i++) {

```

```

        if (!memory[i].free && memory[i].process_id == pid) {
            memory[i].free = 1;
            memory[i].process_id = -1;
            return;
        }
    }
}

/* ===== FIRST FIT ===== */
int first_fit(int pid, int size) {
    for (int i = 0; i < block_count; i++) {
        if (memory[i].free && memory[i].size >= size) {

            if (memory[i].size > size) {
                for (int j = block_count; j > i + 1; j--)
                    memory[j] = memory[j - 1];

                memory[i + 1].size = memory[i].size - size;
                memory[i + 1].free = 1;
                memory[i + 1].process_id = -1;
                block_count++;
            }

            memory[i].size = size;
            memory[i].free = 0;
            memory[i].process_id = pid;
            return 1;
        }
    }
    return 0;
}

/* ===== BEST FIT ===== */

```

```

int best_fit(int pid, int size) {
    int best = -1;

    for (int i = 0; i < block_count; i++) {
        if (memory[i].free && memory[i].size >= size) {
            if (best == -1 || memory[i].size < memory[best].size)
                best = i;
        }
    }

    if (best == -1) return 0;

    if (memory[best].size > size) {
        for (int j = block_count; j > best + 1; j--)
            memory[j] = memory[j - 1];

        memory[best + 1].size = memory[best].size - size;
        memory[best + 1].free = 1;
        memory[best + 1].process_id = -1;
        block_count++;
    }

    memory[best].size = size;
    memory[best].free = 0;
    memory[best].process_id = pid;
    return 1;
}

/* ===== WORST FIT ===== */
int worst_fit(int pid, int size) {
    int worst = -1;

    for (int i = 0; i < block_count; i++) {

```

```

    if (memory[i].free && memory[i].size >= size) {
        if (worst == -1 || memory[i].size > memory[worst].size)
            worst = i;
    }
}

if (worst == -1) return 0;

if (memory[worst].size > size) {
    for (int j = block_count; j > worst + 1; j--)
        memory[j] = memory[j - 1];

    memory[worst + 1].size = memory[worst].size - size;
    memory[worst + 1].free = 1;
    memory[worst + 1].process_id = -1;
    block_count++;
}

memory[worst].size = size;
memory[worst].free = 0;
memory[worst].process_id = pid;
return 1;
}

/* ===== NEXT FIT ===== */
int next_fit(int pid, int size) {
    int i = last_alloc_index;
    int scanned = 0;

    while (scanned < block_count) {
        if (memory[i].free && memory[i].size >= size) {

            if (memory[i].size > size) {

```

```

        for (int j = block_count; j > i + 1; j--)
            memory[j] = memory[j - 1];

        memory[i + 1].size = memory[i].size - size;
        memory[i + 1].free = 1;
        memory[i + 1].process_id = -1;
        block_count++;
    }

    memory[i].size = size;
    memory[i].free = 0;
    memory[i].process_id = pid;

    last_alloc_index = i;
    return 1;
}

i = (i + 1) % block_count;
scanned++;
}
return 0;
}

/* ===== FRAGMENTATION ===== */
void calculate_fragmentation() {
    int total_free = 0, largest = 0, holes = 0;

    for (int i = 0; i < block_count; i++) {
        if (memory[i].free) {
            total_free += memory[i].size;
            holes++;
            if (memory[i].size > largest)
                largest = memory[i].size;
        }
    }
}

```

```

    }
}

float ext = (total_free > 0) ?
    (1.0 - (float)largest / total_free) * 100 : 0;

printf("Total Free Space    : %d\n", total_free);
printf("Largest Free Block   : %d\n", largest);
printf("Free Holes           : %d\n", holes);
printf("External Fragmentation: %.2f%%\n", ext);
}

/* ===== BENCHMARK ===== */
void benchmark(const char *name, int (*alloc)(int,int)) {
    init_memory(1000);
    clock_t start = clock();

    alloc(1,100);
    alloc(2,200);
    alloc(3,300);
    alloc(4,250);
    alloc(5,150);

    deallocate(2);
    deallocate(4);

    alloc(6,180);
    alloc(7,220);
    alloc(8,100);

    clock_t end = clock();

    printf("\n=== %s ===\n", name);

```

```

printf("Execution Time: %.2f us\n",
      (double)(end-start)*1e6/CLOCKS_PER_SEC);

calculate_fragmentation();
display_memory();
}

/* ===== MAIN ===== */

int main() {
    benchmark("First Fit", first_fit);
    benchmark("Best Fit", best_fit);
    benchmark("Worst Fit", worst_fit);
    benchmark("Next Fit", next_fit);
    return 0;
}

```

3.2.3 Screenshot Output

```

danny@LAPTOP-GRJ2V1D:~/praktikum-memory/contiguous-allocation$ ./memory_allocator

=== First Fit ===
Execution Time: 11.00 us
Total Free Space : 50
Largest Free Block : 30
Free Holes : 2
External Fragmentation: 40.00%

Memory State:
Block 0: P1 | Size = 100
Block 1: P6 | Size = 100
Block 2: FREE | Size = 20
Block 3: P3 | Size = 300
Block 4: P7 | Size = 220
Block 5: FREE | Size = 30
Block 6: P5 | Size = 150

=== Best Fit ===
Execution Time: 3.00 us
Total Free Space : 50
Largest Free Block : 30
Free Holes : 2
External Fragmentation: 40.00%

Memory State:
Block 0: P1 | Size = 100
Block 1: P6 | Size = 100
Block 2: FREE | Size = 20
Block 3: P3 | Size = 300
Block 4: P7 | Size = 220
Block 5: FREE | Size = 30
Block 6: P5 | Size = 150

=== Worst Fit ===
Execution Time: 2.00 us
Total Free Space : 170
Largest Free Block : 100
Free Holes : 2
External Fragmentation: 41.18%

Memory State:
Block 0: P1 | Size = 100
Block 1: P6 | Size = 100
Block 2: FREE | Size = 100
Block 3: P3 | Size = 300
Block 4: P5 | Size = 150
Block 5: FREE | Size = 70
Block 6: P5 | Size = 150

=== Next Fit ===
Execution Time: 2.00 us
Total Free Space : 50
Largest Free Block : 30
Free Holes : 2
External Fragmentation: 40.00%

Memory State:
Block 0: P1 | Size = 100
Block 1: P6 | Size = 100
Block 2: FREE | Size = 20
Block 3: P3 | Size = 300
Block 4: P7 | Size = 220
Block 5: FREE | Size = 30
Block 6: P5 | Size = 150

```

3.2.4 Penjelasan Implementasi

- Preprocessing - Header files (stdio.h, time.h) dimasukkan ke kode

- Compilation - Kode C dengan struct Block dan semua fungsi alokasi dikompilasi ke assembly
- Assembly - Assembly code diubah menjadi object file berisi machine code
- Linking - Object file digabung dengan library C menghasilkan executable file
- Loading - OS memuat executable ke memori:
 - Text Segment: Kode fungsi main, init_memory, display_memory, deallocate, first_fit, best_fit, worst_fit, next_fit, calculate_fragmentation, benchmark
 - Data Segment: Array memory[100] dan variabel block_count, last_alloc_index
- Stack Setup - Frame stack untuk fungsi main() dibuat
- Main Execution - main() mulai dieksekusi, memanggil fungsi benchmark() untuk keempat algoritma
- Memory Initialization - init_memory() mengatur block pertama sebagai free memory 1000 units
- Algorithm Benchmarking - Untuk setiap algoritma:
 - Mengalokasikan 5 proses dengan ukuran berbeda
 - Mendealokasi beberapa proses
 - Mengalokasikan proses baru
 - Mengukur waktu eksekusi dengan clock()
- Fragmentation Calculation - calculate_fragmentation() menghitung:
 - Total free space, largest free block, jumlah free holes
 - External fragmentation percentage
- Memory Display - display_memory() menampilkan status semua block memory
- Performance Comparison - Program membandingkan waktu eksekusi dan fragmentasi untuk setiap algoritma
- Termination - Program selesai dengan return value 0

3.3 TUGAS 3

3.3.1 Deskripsi Tugas

Tugas 3 berfokus pada pengembangan sistem paging dengan menambahkan page replacement algorithm dan analisis statistik. Mahasiswa diminta mengimplementasikan algoritma FIFO dan LRU ketika frame penuh, serta mencatat metrik seperti jumlah page fault, hit ratio, dan disk write. Pengujian dilakukan menggunakan reference string tertentu dengan variasi jumlah frame untuk mengamati fenomena Belady's Anomaly. Selain itu, mahasiswa

menyusun tabel trace detail untuk setiap algoritma dan membandingkan performa FIFO dan LRU. Tugas ini bertujuan memperdalam pemahaman tentang efisiensi penggantian halaman dalam sistem memori virtual.

3.3.2 Source Code

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <math.h>
#include <time.h>
#include <limits.h>

#define FRAME_SIZE 256
#define MAX_FRAMES 16
#define PAGE_SIZE FRAME_SIZE

// Algoritma yang tersedia
typedef enum {
    FIFO,
    LRU
} Algorithm;

typedef struct {
    int page_number;
    int frame_number;
    int valid;
    int reference_bit;
    int modify_bit;
    int last_used; // Untuk LRU - timestamp penggunaan terakhir
} PageTableEntry;

typedef struct {
    int occupied;
    int page_number;
```

```

    int load_time; // Untuk FIFO - waktu pemuatan halaman
    int dirty; // Untuk statistik - halaman telah dimodifikasi
} Frame;

// Variabel global
Frame physical_memory[MAX_FRAMES];
PageTableEntry page_table[100];
int num_pages = 0;
int num_frames = 4; // Default 4 frames untuk test scenario
Algorithm current_algorithm = FIFO;

// Statistik
int page_faults = 0;
int page_hits = 0;
int disk_writes = 0;
int total_accesses = 0;

// Untuk FIFO
int fifo_queue[MAX_FRAMES];
int front = 0, rear = -1, queue_count = 0;

// Untuk LRU
int lru_counter = 0;

void initialize_system(int frames) {
    num_frames = frames;

    // Reset memory frames
    for (int i = 0; i < MAX_FRAMES; i++) {
        physical_memory[i].occupied = 0;
        physical_memory[i].page_number = -1;
        physical_memory[i].load_time = 0;
        physical_memory[i].dirty = 0;
    }
}

```

```

}

// Reset page table
for (int i = 0; i < 100; i++) {
    page_table[i].page_number = i;
    page_table[i].frame_number = -1;
    page_table[i].valid = 0;
    page_table[i].reference_bit = 0;
    page_table[i].modify_bit = 0;
    page_table[i].last_used = -1;
}

// Reset FIFO queue
front = 0;
rear = -1;
queue_count = 0;

// Reset LRU counter
lru_counter = 0;

// Reset statistik
page_faults = 0;
page_hits = 0;
disk_writes = 0;
total_accesses = 0;
}

void display_page_table() {
    printf("\n=== PAGE TABLE ===\n");
    printf("%-12s %-15s %-10s %-12s %-12s %-12s\n",
           "Page", "Frame", "Valid", "Referenced", "Modified", "Last Used");
    printf("-----\n");
}

```

```

for (int i = 0; i < num_pages; i++) {
    printf("%-12d %-15d %-10s %-12s %-12s %-12d\n",
        page_table[i].page_number,
        page_table[i].frame_number,
        page_table[i].valid ? "Yes" : "No",
        page_table[i].reference_bit ? "Yes" : "No",
        page_table[i].modify_bit ? "Yes" : "No",
        page_table[i].last_used);
}
printf("\n");
}

void display_physical_memory() {
    printf("=== PHYSICAL MEMORY (%d FRAMES) ===\n", num_frames);
    printf("%-10s %-10s %-15s %-10s %-10s\n",
        "Frame", "Status", "Page Number", "Load Time", "Dirty");
    printf("-----\n");

    for (int i = 0; i < num_frames; i++) {
        printf("%-10d %-10s", i, physical_memory[i].occupied ? "Occupied" : "Free");
        if (physical_memory[i].occupied) {
            printf("%-15d %-10d %-10s",
                physical_memory[i].page_number,
                physical_memory[i].load_time,
                physical_memory[i].dirty ? "Yes" : "No");
        }
        printf("\n");
    }
    printf("\n");
}

int find_free_frame() {
    for (int i = 0; i < num_frames; i++) {

```

```

        if (!physical_memory[i].occupied) {
            return i;
        }
    }
    return -1;
}

void enqueue_fifo(int page_num) {
    rear = (rear + 1) % num_frames;
    fifo_queue[rear] = page_num;
    if (queue_count < num_frames) {
        queue_count++;
    }
}

int dequeue_fifo() {
    if (queue_count == 0) return -1;
    int page_num = fifo_queue[front];
    front = (front + 1) % num_frames;
    queue_count--;
    return page_num;
}

int find_lru_victim() {
    int victim_frame = -1;
    int min_last_used = INT_MAX;

    for (int i = 0; i < num_frames; i++) {
        if (physical_memory[i].occupied) {
            int page_num = physical_memory[i].page_number;
            if (page_table[page_num].last_used < min_last_used) {
                min_last_used = page_table[page_num].last_used;
                victim_frame = i;
            }
        }
    }
    return victim_frame;
}

```

```

    }
    }
}
return victim_frame;
}

int find_fifo_victim() {
    // FIFO menggunakan queue, jadi kita ambil dari depan
    if (queue_count == 0) return -1;

    int victim_page = fifo_queue[front];
    for (int i = 0; i < num_frames; i++) {
        if (physical_memory[i].occupied && physical_memory[i].page_number ==
victim_page) {
            return i;
        }
    }
    return -1;
}

int replace_page(int page_num, int write) {
    int victim_frame = -1;

    if (current_algorithm == FIFO) {
        victim_frame = find_fifo_victim();
    } else if (current_algorithm == LRU) {
        victim_frame = find_lru_victim();
    }

    if (victim_frame == -1) {
        return -1;
    }
}

```

```
int victim_page = physical_memory[victim_frame].page_number;

// Jika halaman korban telah dimodifikasi, perlu write ke disk
if (page_table[victim_page].modify_bit && physical_memory[victim_frame].dirty) {
    disk_writes++;
    printf(" [DISK WRITE] Page %d (dirty) ditulis ke disk\n", victim_page);
}

// Update page table untuk halaman korban
page_table[victim_page].valid = 0;
page_table[victim_page].frame_number = -1;
page_table[victim_page].modify_bit = 0;

// Update page table untuk halaman baru
page_table[page_num].frame_number = victim_frame;
page_table[page_num].valid = 1;
page_table[page_num].reference_bit = 1;
page_table[page_num].last_used = lru_counter++;
if (write) {
    page_table[page_num].modify_bit = 1;
}

// Update physical memory
physical_memory[victim_frame].page_number = page_num;
physical_memory[victim_frame].load_time = total_accesses;
physical_memory[victim_frame].dirty = write ? 1 : 0;

// Update FIFO queue jika menggunakan FIFO
if (current_algorithm == FIFO) {
    dequeue_fifo();
    enqueue_fifo(page_num);
}
```

```

return victim_frame;
}

int allocate_page(int page_num, int write) {
    total_accesses++;

    if (page_num >= 100) {
        printf("Error: Page number terlalu besar\n");
        return -1;
    }

    // Jika halaman sudah ada di memori (HIT)
    if (page_table[page_num].valid) {
        page_hits++;
        page_table[page_num].reference_bit = 1;
        page_table[page_num].last_used = lru_counter++;

        if (write) {
            page_table[page_num].modify_bit = 1;
            // Update dirty bit di physical memory
            int frame_num = page_table[page_num].frame_number;
            if (frame_num >= 0 && frame_num < num_frames) {
                physical_memory[frame_num].dirty = 1;
            }
        }

        return page_table[page_num].frame_number;
    }

    // PAGE FAULT
    page_faults++;

    int frame = find_free_frame();

```

```

if (frame == -1) {
    // Tidak ada frame kosong, perlu page replacement
    printf(" [PAGE REPLACEMENT] Menggunakan algoritma %s\n",
        current_algorithm == FIFO ? "FIFO" : "LRU");
    frame = replace_page(page_num, write);
    if (frame == -1) {
        printf("Error: Gagal melakukan page replacement\n");
        return -1;
    }
    printf(" Page %d menggantikan page di frame %d\n", page_num, frame);
} else {
    // Ada frame kosong
    page_table[page_num].frame_number = frame;
    page_table[page_num].valid = 1;
    page_table[page_num].reference_bit = 1;
    page_table[page_num].last_used = lru_counter++;
    if (write) {
        page_table[page_num].modify_bit = 1;
    }

    physical_memory[frame].occupied = 1;
    physical_memory[frame].page_number = page_num;
    physical_memory[frame].load_time = total_accesses;
    physical_memory[frame].dirty = write ? 1 : 0;

    // Untuk FIFO, tambahkan ke queue
    if (current_algorithm == FIFO) {
        enqueue_fifo(page_num);
    }

    printf("Page %d dialokasikan ke frame %d\n", page_num, frame);
}

```

```

    if (page_num >= num_pages) {
        num_pages = page_num + 1;
    }

    return frame;
}

void display_statistics() {
    printf("\n=== STATISTIK ===\n");
    printf("Algoritma: %s\n", current_algorithm == FIFO ? "FIFO" : "LRU");
    printf("Jumlah Frames: %d\n", num_frames);
    printf("Total Akses: %d\n", total_accesses);
    printf("Page Faults: %d\n", page_faults);
    printf("Page Hits: %d\n", page_hits);

    if (total_accesses > 0) {
        float fault_rate = (float)page_faults / total_accesses * 100;
        float hit_rate = (float)page_hits / total_accesses * 100;
        printf("Page Fault Rate: %.2f%%\n", fault_rate);
        printf("Hit Rate: %.2f%%\n", hit_rate);
    }

    printf("Disk Writes: %d\n", disk_writes);
    printf("\n");
}

void run_test_scenario(int frames, Algorithm algo) {

    printf("\n=====
=====
    printf("|| TEST SCENARIO - %s dengan %d frames ||\n",
        algo == FIFO ? "FIFO" : "LRU", frames);
}

```

```

printf("=====
=====\\n\\n");

initialize_system(frames);
current_algorithm = algo;

// Reference string dari soal
int reference_string[] = {7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,7,0,1};
int length = sizeof(reference_string) / sizeof(reference_string[0]);

// Untuk menampilkan trace table
printf("Step\tReference\tFrame 0\tFrame 1\tFrame 2\tFrame 3\tFault?\n");
printf("-----\n");

for (int i = 0; i < length; i++) {
    int page_num = reference_string[i];
    int fault_before = page_faults;

    // Simulasi akses read
    allocate_page(page_num, 0);

    // Tampilkan trace
    printf("%d\t%d\t", i+1, page_num);

    // Tampilkan isi frames
    for (int f = 0; f < frames; f++) {
        if (physical_memory[f].occupied) {
            printf("%d\t", physical_memory[f].page_number);
        } else {
            printf("-\t");
        }
    }
}

```

```
// Tampilkan apakah terjadi fault
if (page_faults > fault_before) {
    printf("FAULT");
} else {
    printf("HIT");
}
printf("\n");
}

display_statistics();
}

void compare_algorithms() {

printf("\n=====
=====\\n");

printf("|| PERBANDINGAN ALGORITMA (4 frames) ||\\n");

printf("||=====
=====\\n\\n");

printf("%-15s %-20s %-15s %-20s\\n",
        "Algoritma", "Jumlah Page Fault", "Hit Ratio", "Disk Writes");
printf("-----\\n");

// Jalankan FIFO
run_test_scenario(4, FIFO);
int fifo_faults = page_faults;
int fifo_writes = disk_writes;
float fifo_hit_rate = (float)page_hits / total_accesses * 100;

// Jalankan LRU
```

```

run_test_scenario(4, LRU);

int lru_faults = page_faults;
int lru_writes = disk_writes;
float lru_hit_rate = (float)page_hits / total_accesses * 100;

printf("\n=== TABEL PERBANDINGAN ===\n");
printf("%-15s %-20d %-15.2f%% %-20d\n",
        "FIFO", fifo_faults, fifo_hit_rate, fifo_writes);
printf("%-15s %-20d %-15.2f%% %-20d\n",
        "LRU", lru_faults, lru_hit_rate, lru_writes);
}

void belady_anomaly_test() {

printf("\n=====
=====
\");
printf("|| BELADY'S ANOMALY TEST ||\n");

printf("=====
=====
\");

printf("Menguji apakah page fault berkurang seiring bertambahnya frames:\n\n");

// Test untuk 3, 4, dan 5 frames
int frames_list[] = {3, 4, 5};
int num_tests = sizeof(frames_list) / sizeof(frames_list[0]);

printf("%-10s %-15s %-15s\n", "Frames", "FIFO Faults", "LRU Faults");
printf("-----\n");

for (int i = 0; i < num_tests; i++) {
    int frames = frames_list[i];

```

```

// Test FIFO
run_test_scenario(frames, FIFO);
int fifo_faults = page_faults;

// Test LRU
run_test_scenario(frames, LRU);
int lru_faults = page_faults;

printf("%-10d %-15d %-15d\n", frames, fifo_faults, lru_faults);
}

printf("\nANALISIS:\n");
printf("Belady's Anomaly terjadi ketika jumlah page fault BERTAMBAH\n");
printf("meskipun jumlah frame ditambah. Fenomena ini bisa terjadi\n");
printf("pada algoritma FIFO tetapi tidak pada LRU.\n");
}

void demo_scenario() {

printf("\n=====
=====
    printf("    DEMO SCENARIO: PAGING SYSTEM    \n");

printf("=====
=====
    \n\n");

    initialize_system(4);
    current_algorithm = FIFO;

    printf("Skenario: Program berukuran 2048 bytes\n");
    printf("Jumlah pages: %d pages\n\n", 2048 / PAGE_SIZE);

    // Simulasi beberapa akses

```

```

int accesses[][2] = {
    {0, 0}, // Read address 0
    {300, 1}, // Write address 300
    {512, 0}, // Read address 512
    {100, 0}, // Read address 100
    {1024, 1} // Write address 1024
};

for (int i = 0; i < 5; i++) {
    int addr = accesses[i][0];
    int write = accesses[i][1];

    printf("\nAkses %s pada logical address %d:\n",
        write ? "WRITE" : "READ", addr);

    int page_num = addr / PAGE_SIZE;
    int frame = allocate_page(page_num, write);

    if (frame != -1) {
        printf(" Page %d -> Frame %d\n", page_num, frame);
    }
}

display_page_table();
display_physical_memory();
display_statistics();
}

int main() {
    int choice, address, write, frames, algo_choice;

    // Default initialization
    initialize_system(4);

```

```

while(1) {
    printf("\n=== ADVANCED PAGING SIMULATOR ===\n");
    printf("1. Set Jumlah Frames\n");
    printf("2. Set Algoritma (FIFO/LRU)\n");
    printf("3. Allocate Page\n");
    printf("4. Access Memory (Read)\n");
    printf("5. Access Memory (Write)\n");
    printf("6. Display Page Table\n");
    printf("7. Display Physical Memory\n");
    printf("8. Display Statistics\n");
    printf("9. Run Demo Scenario\n");
    printf("10. Run Test Scenario (Reference String)\n");
    printf("11. Compare FIFO vs LRU\n");
    printf("12. Belady's Anomaly Test\n");
    printf("13. Reset System\n");
    printf("14. Exit\n");
    printf("Pilihan: ");
    scanf("%d", &choice);

    switch(choice) {
        case 1:
            printf("Masukkan jumlah frames (1-%d): ", MAX_FRAMES);
            scanf("%d", &frames);
            if (frames > 0 && frames <= MAX_FRAMES) {
                initialize_system(frames);
                printf("Sistem direset dengan %d frames\n", frames);
            } else {
                printf("Jumlah frames tidak valid!\n");
            }
            break;

        case 2:

```

```
printf("Pilih algoritma:\n");
printf("1. FIFO\n");
printf("2. LRU\n");
printf("Pilihan: ");
scanf("%d", &algo_choice);
if (algo_choice == 1) {
    current_algorithm = FIFO;
    printf("Algoritma diubah menjadi FIFO\n");
} else if (algo_choice == 2) {
    current_algorithm = LRU;
    printf("Algoritma diubah menjadi LRU\n");
} else {
    printf("Pilihan tidak valid!\n");
}
break;
```

case 3:

```
printf("Masukkan page number: ");
scanf("%d", &address);
printf("Tipe akses (0=Read, 1=Write): ");
scanf("%d", &write);
allocate_page(address, write);
break;
```

case 4:

```
printf("Masukkan logical address: ");
scanf("%d", &address);
printf("Page %d diakses (READ)\n", address / PAGE_SIZE);
allocate_page(address / PAGE_SIZE, 0);
break;
```

case 5:

```
printf("Masukkan logical address: ");
```

```

scanf("%d", &address);
printf("Page %d diakses (WRITE)\n", address / PAGE_SIZE);
allocate_page(address / PAGE_SIZE, 1);
break;

case 6:
    display_page_table();
    break;

case 7:
    display_physical_memory();
    break;

case 8:
    display_statistics();
    break;

case 9:
    demo_scenario();
    break;

case 10:
    printf("Masukkan jumlah frames untuk test: ");
    scanf("%d", &frames);
    printf("Pilih algoritma (1=FIFO, 2=LRU): ");
    scanf("%d", &algo_choice);
    if (frames > 0 && frames <= MAX_FRAMES && (algo_choice == 1 ||
algo_choice == 2)) {
        run_test_scenario(frames, algo_choice == 1 ? FIFO : LRU);
    } else {
        printf("Input tidak valid!\n");
    }
    break;

```

```
case 11:
    compare_algorithms();
    break;

case 12:
    belady_anomaly_test();
    break;

case 13:
    initialize_system(num_frames);
    printf("Sistem direset\n");
    break;

case 14:
    printf("Keluar dari program...\n");
    return 0;

default:
    printf("Pilihan tidak valid!\n");
}
}

return 0;
}
```

3.3.3 Screenshot Output

```
#####AFTW-08.3.2010-/-praktikum-memory/paging$ ./paging_simulator

=== ADVANCED PAGING SIMULATOR ===
1. Set Jumlah Frames
2. Set Algoritma (FIFO/LRU)
3. Allocate Page
4. Access Memory (Read)
5. Access Memory (Write)
6. Display Page Table
7. Display Physical Memory
8. Display Statistics
9. Run Demo Scenario
10. Run Test Scenario (Reference String)
11. Compare FIFO vs LRU
12. Delay's Anomaly Test
13. Reset System
14. Exit
Pilihan: 10
Masukkan jumlah frames untuk test: 4
Pilih algoritma (1=FIFO, 2=LRU): 1

TEST SCENARIO - FIFO dengan 4 frames

Step  Reference      Frame 0 Frame 1 Frame 2 Frame 3 Fault?
Page 7 dialokasikan ke frame 0
1      7      -      -      -      -      FAULT
Page 8 dialokasikan ke frame 1
2      8      7      -      -      -      FAULT
Page 1 dialokasikan ke frame 2
3      1      7      0      1      -      FAULT
Page 2 dialokasikan ke frame 3
4      2      7      0      1      2      FAULT
5      0      7      0      1      2      HIT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO
Page 3 menggantikan page di frame 0
6      3      0      0      1      2      FAULT
7      0      3      0      1      2      HIT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO
Page 4 menggantikan page di frame 1
8      4      3      4      1      2      FAULT
9      2      3      4      1      2      HIT
10     0      3      4      1      2      HIT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO
Page 8 menggantikan page di frame 2
11     8      3      4      0      2      FAULT
12     3      3      4      0      2      HIT
13     0      3      4      0      2      HIT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO
Page 1 menggantikan page di frame 3
14     1      3      4      0      1      FAULT
15     2      3      4      0      1      FAULT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO
Page 2 menggantikan page di frame 0
16     0      2      4      0      1      HIT
17     1      2      4      0      1      HIT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO
Page 7 menggantikan page di frame 1
18     7      2      7      0      1      FAULT
19     0      2      7      0      1      HIT
20     1      2      7      0      1      HIT

=== STATISTIK ===
Algoritma: FIFO
Jumlah Frames: 4
Total Akses: 20
Page Faults: 10
Page Hits: 10
Page Fault Rate: 50.00%
Hit Rate: 50.00%
Disk Writes: 0
```

```
TEST SCENARIO - LRU dengan 4 frames

Step  Reference      Frame 0 Frame 1 Frame 2 Frame 3 Fault?
Page 7 dialokasikan ke frame 0
1      7      -      -      -      -      FAULT
Page 8 dialokasikan ke frame 1
2      8      7      -      -      -      FAULT
Page 1 dialokasikan ke frame 2
3      1      7      0      1      -      FAULT
Page 2 dialokasikan ke frame 3
4      2      7      0      1      2      FAULT
5      0      7      0      1      2      HIT
[PAGE REPLACEMENT] Menggunakan algoritma LRU
Page 3 menggantikan page di frame 0
6      3      0      1      2      2      FAULT
7      0      3      0      2      2      HIT
[PAGE REPLACEMENT] Menggunakan algoritma LRU
Page 4 menggantikan page di frame 2
8      4      3      0      4      2      FAULT
9      2      3      0      4      2      HIT
10     0      3      0      4      2      HIT
11     8      3      0      4      2      HIT
12     3      3      0      4      2      HIT
13     2      3      0      4      2      HIT
[PAGE REPLACEMENT] Menggunakan algoritma LRU
Page 1 menggantikan page di frame 2
14     1      3      0      1      2      FAULT
15     2      3      0      1      2      HIT
16     0      3      0      1      2      HIT
17     1      3      0      1      2      HIT
[PAGE REPLACEMENT] Menggunakan algoritma LRU
Page 7 menggantikan page di frame 0
18     7      0      1      2      2      FAULT
19     0      7      0      1      2      HIT
20     1      7      0      1      2      HIT

=== STATISTIK ===
Algoritma: LRU
Jumlah Frames: 4
Total Akses: 20
Page Faults: 8
Page Hits: 12
Page Fault Rate: 40.00%
Hit Rate: 60.00%
Disk Writes: 0
```

TEST SCENARIO - FIFO dengan 3 frames

Step	Reference	Frame 0	Frame 1	Frame 2	Frame 3	Fault?
Page 7 dialokasikan ke frame 0						
1	7	-	-	-	-	FAULT
Page 0 dialokasikan ke frame 1						
2	0	7	0	-	-	FAULT
Page 1 dialokasikan ke frame 2						
3	1	7	0	1	-	FAULT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO						
Page 2 menggantikan page di frame 0						
4	2	2	0	1	-	FAULT
5	0	2	0	1	-	HIT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO						
Page 3 menggantikan page di frame 1						
6	3	2	3	1	-	FAULT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO						
Page 0 menggantikan page di frame 2						
7	0	2	3	0	-	FAULT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO						
Page 4 menggantikan page di frame 0						
8	4	4	3	0	-	FAULT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO						
Page 2 menggantikan page di frame 1						
9	2	4	2	0	-	FAULT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO						
Page 3 menggantikan page di frame 2						
10	3	4	2	3	-	FAULT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO						
Page 0 menggantikan page di frame 0						
11	0	0	2	3	-	FAULT
12	3	0	2	3	-	HIT
13	2	0	2	3	-	HIT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO						
Page 1 menggantikan page di frame 1						
14	1	0	1	3	-	FAULT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO						
Page 2 menggantikan page di frame 2						
15	2	0	1	2	-	FAULT
16	0	0	1	2	-	HIT
17	1	0	1	2	-	HIT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO						
Page 7 menggantikan page di frame 0						
18	7	7	1	2	-	FAULT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO						
Page 0 menggantikan page di frame 1						
19	0	0	0	2	-	FAULT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO						
Page 1 menggantikan page di frame 2						
20	1	7	0	1	-	FAULT
=== STATISTIK ===						
Algoritma: FIFO						
Jumlah Frames: 3						
Total Akses: 20						
Page Faults: 15						
Page Hits: 5						
Page Fault Rate: 75.00%						
Hit Rate: 25.00%						
Disk Writes: 0						

TEST SCENARIO - LRU dengan 3 frames

Step	Reference	Frame 0	Frame 1	Frame 2	Frame 3	Fault?
Page 7 dialokasikan ke frame 0						
1	7	-	-	-	-	FAULT
Page 0 dialokasikan ke frame 1						
2	0	7	0	-	-	FAULT
Page 1 dialokasikan ke frame 2						
3	1	7	0	1	-	FAULT
[PAGE REPLACEMENT] Menggunakan algoritma LRU						
Page 2 menggantikan page di frame 0						
4	2	2	0	1	-	FAULT
5	0	2	0	1	-	HIT
[PAGE REPLACEMENT] Menggunakan algoritma LRU						
Page 3 menggantikan page di frame 2						
6	3	2	0	3	-	FAULT
7	0	2	0	3	-	HIT
[PAGE REPLACEMENT] Menggunakan algoritma LRU						
Page 4 menggantikan page di frame 0						
8	4	4	0	3	-	FAULT
[PAGE REPLACEMENT] Menggunakan algoritma LRU						
Page 2 menggantikan page di frame 2						
9	2	4	0	2	-	FAULT
[PAGE REPLACEMENT] Menggunakan algoritma LRU						
Page 3 menggantikan page di frame 1						
10	3	4	3	2	-	FAULT
[PAGE REPLACEMENT] Menggunakan algoritma LRU						
Page 0 menggantikan page di frame 0						
11	0	0	3	2	-	FAULT
12	3	0	3	2	-	HIT
13	2	0	3	2	-	HIT
[PAGE REPLACEMENT] Menggunakan algoritma LRU						
Page 1 menggantikan page di frame 0						
14	1	1	3	2	-	FAULT
15	2	1	3	2	-	HIT
[PAGE REPLACEMENT] Menggunakan algoritma LRU						
Page 0 menggantikan page di frame 1						
16	0	1	0	2	-	FAULT
17	1	1	0	2	-	HIT
[PAGE REPLACEMENT] Menggunakan algoritma LRU						
Page 7 menggantikan page di frame 2						
18	7	1	0	7	-	FAULT
19	0	1	0	7	-	HIT
20	1	1	0	7	-	HIT
=== STATISTIK ===						
Algoritma: LRU						
Jumlah Frames: 3						
Total Akses: 20						
Page Faults: 12						
Page Hits: 8						
Page Fault Rate: 60.00%						
Hit Rate: 40.00%						
Disk Writes: 0						

TEST SCENARIO - FIFO dengan 4 frames					
Step	Reference	Frame 0	Frame 1	Frame 2	Frame 3 Fault?
1	Page 7 dialokasikan ke frame 0	-	-	-	FAULT
2	Page 0 dialokasikan ke frame 1	0	-	-	FAULT
3	Page 1 dialokasikan ke frame 2	0	1	-	FAULT
4	Page 2 dialokasikan ke frame 3	0	1	2	FAULT
5	[PAGE REPLACEMENT] Menggunakan algoritma FIFO	0	1	2	HIT
6	Page 3 menggantikan page di frame 0	3	1	2	FAULT
7	[PAGE REPLACEMENT] Menggunakan algoritma FIFO	3	0	1	2
8	Page 4 menggantikan page di frame 1	3	4	1	2
9	[PAGE REPLACEMENT] Menggunakan algoritma FIFO	3	4	1	2
10	Page 0 menggantikan page di frame 2	3	4	0	2
11	[PAGE REPLACEMENT] Menggunakan algoritma FIFO	3	4	0	2
12	Page 3 menggantikan page di frame 3	3	4	0	2
13	[PAGE REPLACEMENT] Menggunakan algoritma FIFO	3	4	0	2
14	Page 1 menggantikan page di frame 0	1	4	0	1
15	[PAGE REPLACEMENT] Menggunakan algoritma FIFO	1	4	0	1
16	Page 2 menggantikan page di frame 1	1	2	0	1
17	[PAGE REPLACEMENT] Menggunakan algoritma FIFO	1	2	0	1
18	Page 7 menggantikan page di frame 2	1	2	7	0
19	[PAGE REPLACEMENT] Menggunakan algoritma FIFO	1	2	7	0
20	Page 0 menggantikan page di frame 3	1	2	7	0
=== STATISTIK ===					
Algoritma: FIFO					
Jumlah Frames: 4					
Total Akses: 20					
Page Faults: 18					
Page Hits: 18					
Page Fault Rate: 90.00%					
Hit Rate: 10.00%					
Disk Writes: 0					
TEST SCENARIO - LRU dengan 4 frames					
Step	Reference	Frame 0	Frame 1	Frame 2	Frame 3 Fault?
1	Page 7 dialokasikan ke frame 0	7	-	-	FAULT
2	Page 0 dialokasikan ke frame 1	7	0	-	FAULT
3	Page 1 dialokasikan ke frame 2	7	0	1	FAULT
4	Page 2 dialokasikan ke frame 3	7	0	1	2
5	[PAGE REPLACEMENT] Menggunakan algoritma LRU	7	0	1	2
6	Page 3 menggantikan page di frame 0	3	0	1	2
7	[PAGE REPLACEMENT] Menggunakan algoritma LRU	3	0	1	2
8	Page 4 menggantikan page di frame 2	3	0	4	2
9	[PAGE REPLACEMENT] Menggunakan algoritma LRU	3	0	4	2
10	Page 0 menggantikan page di frame 1	3	0	4	2
11	[PAGE REPLACEMENT] Menggunakan algoritma LRU	3	0	4	2
12	Page 1 menggantikan page di frame 2	3	0	4	2
13	[PAGE REPLACEMENT] Menggunakan algoritma LRU	3	0	4	2
14	Page 1 menggantikan page di frame 3	3	0	1	2
15	[PAGE REPLACEMENT] Menggunakan algoritma LRU	3	0	1	2
16	Page 0 menggantikan page di frame 0	0	0	1	2
17	[PAGE REPLACEMENT] Menggunakan algoritma LRU	0	0	1	2
18	Page 7 menggantikan page di frame 1	0	7	1	2
19	[PAGE REPLACEMENT] Menggunakan algoritma LRU	0	7	1	2
20	Page 1 menggantikan page di frame 2	0	7	1	2
=== STATISTIK ===					
Algoritma: LRU					
Jumlah Frames: 4					
Total Akses: 20					
Page Faults: 8					
Page Hits: 12					
Page Fault Rate: 40.00%					
Hit Rate: 60.00%					
Disk Writes: 0					
4	18	0			

TEST SCENARIO - FIFO dengan 5 frames					
Step	Reference	Frame 0	Frame 1	Frame 2	Frame 3 Fault?
1	Page 7 dialokasikan ke frame 0	7	-	-	-
2	Page 0 dialokasikan ke frame 1	7	0	-	-
3	Page 1 dialokasikan ke frame 2	7	0	1	-
4	Page 2 dialokasikan ke frame 3	7	0	1	2
5	Page 3 dialokasikan ke frame 4	7	0	1	2
6	[PAGE REPLACEMENT] Menggunakan algoritma FIFO	7	0	1	2
7	Page 4 menggantikan page di frame 0	4	0	1	2
8	[PAGE REPLACEMENT] Menggunakan algoritma FIFO	4	0	1	2
9	Page 0 menggantikan page di frame 1	4	0	1	2
10	[PAGE REPLACEMENT] Menggunakan algoritma FIFO	4	0	1	2
11	Page 1 menggantikan page di frame 2	4	0	1	2
12	[PAGE REPLACEMENT] Menggunakan algoritma FIFO	4	0	1	2
13	Page 2 menggantikan page di frame 3	4	0	1	2
14	[PAGE REPLACEMENT] Menggunakan algoritma FIFO	4	0	1	2
15	Page 3 menggantikan page di frame 4	4	0	1	2
16	[PAGE REPLACEMENT] Menggunakan algoritma FIFO	4	0	1	2
17	Page 7 menggantikan page di frame 0	7	0	1	2
18	[PAGE REPLACEMENT] Menggunakan algoritma FIFO	7	0	1	2
19	Page 0 menggantikan page di frame 1	7	0	1	2
20	[PAGE REPLACEMENT] Menggunakan algoritma FIFO	7	0	1	2
=== STATISTIK ===					
Algoritma: FIFO					
Jumlah Frames: 5					
Total Akses: 20					
Page Faults: 9					
Page Hits: 11					
Page Fault Rate: 45.00%					
Hit Rate: 55.00%					
Disk Writes: 0					
TEST SCENARIO - LRU dengan 5 frames					
Step	Reference	Frame 0	Frame 1	Frame 2	Frame 3 Fault?
1	Page 7 dialokasikan ke frame 0	7	-	-	-
2	Page 0 dialokasikan ke frame 1	7	0	-	-
3	Page 1 dialokasikan ke frame 2	7	0	1	-
4	Page 2 dialokasikan ke frame 3	7	0	1	2
5	Page 3 dialokasikan ke frame 4	7	0	1	2
6	[PAGE REPLACEMENT] Menggunakan algoritma LRU	7	0	1	2
7	Page 4 menggantikan page di frame 0	4	0	1	2
8	[PAGE REPLACEMENT] Menggunakan algoritma LRU	4	0	1	2
9	Page 0 menggantikan page di frame 1	4	0	1	2
10	[PAGE REPLACEMENT] Menggunakan algoritma LRU	4	0	1	2
11	Page 1 menggantikan page di frame 2	4	0	1	2
12	[PAGE REPLACEMENT] Menggunakan algoritma LRU	4	0	1	2
13	Page 2 menggantikan page di frame 3	4	0	1	2
14	[PAGE REPLACEMENT] Menggunakan algoritma LRU	4	0	1	2
15	Page 3 menggantikan page di frame 4	4	0	1	2
16	[PAGE REPLACEMENT] Menggunakan algoritma LRU	4	0	1	2
17	Page 7 menggantikan page di frame 0	7	0	1	2
18	[PAGE REPLACEMENT] Menggunakan algoritma LRU	7	0	1	2
19	Page 0 menggantikan page di frame 1	7	0	1	2
20	[PAGE REPLACEMENT] Menggunakan algoritma LRU	7	0	1	2
=== STATISTIK ===					
Algoritma: LRU					
Jumlah Frames: 5					
Total Akses: 20					
Page Faults: 7					
Page Hits: 13					
Page Fault Rate: 35.00%					
Hit Rate: 65.00%					
Disk Writes: 0					
5	9	7			

3.3.4 Penjelasan Implementasi

- Preprocessing - Header files (stdio.h, stdlib.h, string.h, math.h, time.h, limits.h) dimasukkan, semua makro didefinisikan
- Compilation - Kode C kompleks dengan dua algoritma page replacement (FIFO dan LRU) dikompilasi ke assembly
- Assembly - Assembly code diubah menjadi object file berisi machine code untuk simulasi paging lanjutan
- Linking - Object file digabung dengan library C dan libm untuk fungsi matematika, menghasilkan executable file
- Loading - OS memuat executable ke memori:
 - Text Segment: Kode semua fungsi termasuk algoritma FIFO dan LRU
 - Data Segment: Array `physical_memory[16]`, `page_table[100]`, queue FIFO, variabel statistik
- Stack Setup - Frame stack untuk `main()` dibuat
- System Initialization - `initialize_system()` mengatur:
 - Physical memory frames (4 frame default untuk testing)
 - Page table dengan 100 entri (semua invalid)
 - Statistik: `page_faults = 0`, `page_hits = 0`, `disk_writes = 0`
- Algorithm Selection - User memilih algoritma: FIFO (First-In-First-Out) atau LRU (Least Recently Used)
- FIFO Implementation - Menggunakan queue:
 - `enqueue_fifo()` menambah page ke queue
 - `dequeue_fifo()` menghapus page dari queue
 - `find_fifo_victim()` memilih victim berdasarkan urutan masuk
- LRU Implementation - Menggunakan timestamp:
 - Setiap akses memperbarui `last_used` counter
 - `find_lru_victim()` mencari page dengan `last_used` terkecil
- Page Access Simulation - `allocate_page()` menangani:
 - Jika page valid → PAGE HIT: perbarui `last_used` (LRU) atau biarkan queue (FIFO)
 - Jika page invalid → PAGE FAULT: cari frame kosong atau lakukan replacement
- Page Replacement - `replace_page()`:
 - Memilih victim berdasarkan algoritma (FIFO atau LRU)
 - Jika victim dirty (modified), hitung `disk_writes++`

- Ganti victim dengan page baru
- Test Scenario - run_test_scenario():
 - Menggunakan reference string: 7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,7,0,1
 - Menampilkan trace table step-by-step
 - Menghitung page faults untuk setiap konfigurasi
- Belady's Anomaly Test - belady_anomaly_test():
 - Menjalankan simulasi dengan 3, 4, dan 5 frames
 - Membandingkan hasil FIFO dan LRU
 - Demonstrasi fenomena Belady: page fault bisa bertambah meski frames ditambah (pada FIFO)
- Statistics Collection - display_statistics() menampilkan:
 - Total accesses, page faults, page hits
 - Hit rate dan fault rate dalam persen
 - Jumlah disk writes (untuk dirty pages)
- Performance Comparison - compare_algorithms():
 - Membandingkan FIFO vs LRU dengan 4 frames
 - Menunjukkan keunggulan LRU dalam mengurangi page faults
- Main Menu - Program menampilkan menu interaktif dengan 14 opsi
- Termination - Program exit, semua memori simulasi dibersihkan

3.4 TUGAS 4

3.4.1 Deskripsi Tugas

Tugas 4 bertujuan menganalisis dan membandingkan berbagai struktur page table, termasuk three-level page table dan inverted page table. Mahasiswa diminta mengimplementasikan struktur tersebut dan menghitung overhead memori, kecepatan translasi alamat, serta pengaruh penggunaan TLB. Hasil analisis disajikan dalam tabel perbandingan yang mencakup jumlah entri, ukuran memori, dan efisiensi ruang. Selain itu, mahasiswa menganalisis kecocokan masing-masing struktur untuk sistem 32-bit dan 64-bit. Tugas ini menekankan pemahaman desain struktur data memori pada sistem modern.

3.4.2 Source Code

```
#include <stdio.h>
#include <stdlib.h>
```

```

#define L1_BITS 16
#define L2_BITS 8
#define L3_BITS 8
#define OFFSET_BITS 12

#define NUM_FRAMES 1024 // untuk inverted page table

/* ===== */
/* THREE-LEVEL PAGE TABLE */
/* ===== */

typedef struct {
    int frame_number;
    int valid;
    int read, write, execute;
} L3_Entry;

typedef struct {
    L3_Entry *entries;
    int valid;
} L2_Entry;

typedef struct {
    L2_Entry *entries;
    int valid;
} L1_Entry;

L1_Entry *L1;

/* Ambil index dari virtual address */
unsigned int get_l1(unsigned long va) {
    return (va >> (L2_BITS + L3_BITS + OFFSET_BITS)) & ((1 << L1_BITS) - 1);

```

```

}

unsigned int get_l2(unsigned long va) {
    return (va >> (L3_BITS + OFFSET_BITS)) & ((1 << L2_BITS) - 1);
}

unsigned int get_l3(unsigned long va) {
    return (va >> OFFSET_BITS) & ((1 << L3_BITS) - 1);
}

/* Alokasi page */
void allocate_3level(unsigned long va, int frame) {
    int l1 = get_l1(va);
    int l2 = get_l2(va);
    int l3 = get_l3(va);

    if (!L1[l1].valid) {
        L1[l1].entries = calloc(1 << L2_BITS, sizeof(L2_Entry));
        L1[l1].valid = 1;
    }

    if (!L1[l1].entries[l2].valid) {
        L1[l1].entries[l2].entries = calloc(1 << L3_BITS, sizeof(L3_Entry));
        L1[l1].entries[l2].valid = 1;
    }

    L1[l1].entries[l2].entries[l3].frame_number = frame;
    L1[l1].entries[l2].entries[l3].valid = 1;
}

/* Translasi alamat */
int translate_3level(unsigned long va) {
    int l1 = get_l1(va);

```

```

int l2 = get_l2(va);
int l3 = get_l3(va);
int offset = va & ((1 << OFFSET_BITS) - 1);

if (!L1[l1].valid) return -1;
if (!L1[l1].entries[l2].valid) return -1;
if (!L1[l1].entries[l2].entries[l3].valid) return -1;

return (L1[l1].entries[l2].entries[l3].frame_number << OFFSET_BITS) | offset;
}

/* Hitung memory overhead */
void calculate_memory_overhead() {
    int l1_count = 1 << L1_BITS;
    int l2_count = 0, l3_count = 0;

    for (int i = 0; i < l1_count; i++) {
        if (L1[i].valid) {
            l2_count++;
            for (int j = 0; j < (1 << L2_BITS); j++) {
                if (L1[i].entries[j].valid) {
                    l3_count++;
                }
            }
        }
    }
}

printf("\n=== MEMORY OVERHEAD THREE-LEVEL ===\n");
printf("L1 tables : %d\n", l1_count);
printf("L2 tables aktif : %d\n", l2_count);
printf("L3 tables aktif : %d\n", l3_count);
}

```

```

/* ===== */
/* INVERTED PAGE TABLE      */
/* ===== */

typedef struct {
    int process_id;
    int page_number;
    int valid;
} InvertedPageEntry;

InvertedPageEntry inverted_table[NUM_FRAMES];

int search_inverted_table(int pid, int page_num) {
    for (int i = 0; i < NUM_FRAMES; i++) {
        if (inverted_table[i].valid &&
            inverted_table[i].process_id == pid &&
            inverted_table[i].page_number == page_num) {
            return i;
        }
    }
    return -1;
}

void insert_inverted_table(int pid, int page_num, int frame) {
    inverted_table[frame].process_id = pid;
    inverted_table[frame].page_number = page_num;
    inverted_table[frame].valid = 1;
}

/* ===== */
/* MAIN PROGRAM              */
/* ===== */

```

```

int main() {
    unsigned long va1 = 0x123456789ABC;
    unsigned long va2 = 0xABCDEF123456;

    /* Inisialisasi L1 */
    L1 = calloc(1 << L1_BITS, sizeof(L1_Entry));

    /* Alokasi page */
    allocate_3level(va1, 10);
    allocate_3level(va2, 25);

    /* Translasi alamat */
    int pa1 = translate_3level(va1);
    int pa2 = translate_3level(va2);

    printf("Virtual Address 1 : %lx -> Physical Address : %x\n", va1, pa1);
    printf("Virtual Address 2 : %lx -> Physical Address : %x\n", va2, pa2);

    calculate_memory_overhead();

    /* Inverted Page Table */
    insert_inverted_table(1, 5, 100);
    insert_inverted_table(2, 8, 200);

    printf("\n=== INVERTED PAGE TABLE ===\n");
    printf("PID 1 Page 5 -> Frame %d\n",
        search_inverted_table(1, 5));
    printf("PID 2 Page 8 -> Frame %d\n",
        search_inverted_table(2, 8));

    return 0;
}

```

3.4.3 Screenshot Output

```
danyp@LAPTOP-GRJL2V1D:~/praktikum-memory/page-table$ ./three_level_paging
Virtual Address 1 : 123456789abc -> Physical Address : aabc
Virtual Address 2 : abcdef123456 -> Physical Address : 19456

=== MEMORY OVERHEAD THREE-LEVEL ===
L1 tables : 65536
L2 tables aktif : 2
L3 tables aktif : 2

=== INVERTED PAGE TABLE ===
PID 1 Page 5 -> Frame 100
PID 2 Page 8 -> Frame 200
danyp@LAPTOP-GRJL2V1D:~/praktikum-memory/page-table$ |
```

3.4.4 Penjelasan Implementasi

- Preprocessing - Header files (stdio.h, stdlib.h) dimasukkan, semua makro didefinisikan (L1_BITS=16, L2_BITS=8, L3_BITS=8, OFFSET_BITS=12, NUM_FRAMES=1024)
- Compilation - Kode C dengan tiga struct kompleks (L3_Entry, L2_Entry, L1_Entry) dan dua sistem page table berbeda dikompilasi ke assembly
- Assembly - Assembly code diubah menjadi object file berisi machine code untuk simulasi three-level paging dan inverted page table
- Linking - Object file digabung dengan library C menghasilkan executable file
- Loading - OS memuat executable ke memori:
 - Text Segment: Kode fungsi get_l1(), get_l2(), get_l3(), allocate_3level(), translate_3level(), calculate_memory_overhead(), search_inverted_table(), insert_inverted_table(), main()
 - Data Segment: Pointer L1 (awalnya NULL), array inverted_table[1024]
- Stack Setup - Frame stack untuk main() dibuat
- Three-Level Page Table Initialization - main() mengalokasikan L1 table di heap:
 - L1 = calloc(1 << 16, sizeof(L1_Entry)) = 65,536 entri L1
 - Setiap entri L1 berisi pointer ke L2 table (dialokasikan on-demand)
- Address Index Extraction - Fungsi get_l1(), get_l2(), get_l3():
- Virtual address 64-bit: 0x123456789ABC
 - Bit 63-48: L1 index (16 bits)
 - Bit 47-40: L2 index (8 bits)
 - Bit 39-32: L3 index (8 bits)
 - Bit 31-0: Offset (32 bits, tetapi hanya 12-bit offset yang digunakan)
- Three-Level Page Allocation - allocate_3level():
 - Step 1: Cek L1 entry, jika invalid → alokasi L2 table (256 entri)
 - Step 2: Cek L2 entry, jika invalid → alokasi L3 table (256 entri)

- Step 3: Set L3 entry dengan frame number dan valid=1
- Struktur: L1 → L2 tables → L3 tables → Frame numbers
- Three-Level Address Translation - translate_3level():
 - Step 1: Extract L1 index, akses L1 table
 - Step 2: Extract L2 index, akses L2 table (jika L1 valid)
 - Step 3: Extract L3 index, akses L3 entry (jika L2 valid)
 - Step 4: Jika semua valid, hitung: $(\text{frame_number} \ll 12) \mid \text{offset}$
 - Step 5: Jika ada yang invalid → PAGE FAULT
- Memory Overhead Calculation - calculate_memory_overhead():
 - Loop melalui semua 65,536 entri L1
 - Hitung berapa L2 table yang aktif (valid=1)
 - Hitung berapa L3 table yang aktif (valid=1)
 - Tiga-level paging menghemat memori dengan hanya mengalokasikan tabel yang diperlukan
- Inverted Page Table Implementation - Sistem alternatif:
 - Array inverted_table[1024] (satu entri per frame fisik)
 - Setiap entri berisi: process_id, page_number, valid bit
 - Kebalikan dari page table biasa: Frame → Page mapping, bukan Page → Frame
- Inverted Table Operations:
 - insert_inverted_table(): Menyimpan mapping process+page ke frame tertentu
 - search_inverted_table(): Linear search melalui array untuk mencari frame yang berisi page tertentu dari process tertentu
 - Ukuran tabel tetap (1024 entri) = jumlah frame fisik
- Demonstration - Program mendemonstrasikan:
 - Three-level paging dengan alamat virtual 64-bit
 - Alokasi dan translasi dua alamat virtual berbeda
 - Perhitungan overhead memory untuk three-level structure
 - Operasi dasar pada inverted page table
- Cleanup - Program tidak melakukan free() untuk struktur yang dialokasikan di heap (memory leak dalam contoh ini, tetapi akan dibersihkan OS saat program exit)
- Termination - Program selesai, semua memori dibersihkan oleh OS

3.5 TUGAS 5

3.5.1 Deskripsi Tugas

Tugas 5 berfokus pada simulasi mekanisme swapping dengan berbagai strategi pemilihan proses. Mahasiswa diminta mengimplementasikan strategi FIFO, LRU, dan priority-based dalam skenario keterbatasan memori fisik. Simulasi dilakukan dengan sejumlah proses yang melebihi kapasitas memori untuk mengamati perilaku swap in dan swap out. Selanjutnya, mahasiswa menganalisis metrik performa seperti jumlah swap, waktu swap, dan response time. Tugas ini bertujuan memahami dampak swapping terhadap kinerja sistem serta kondisi yang memicu thrashing.

3.5.2 Source Code

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <unistd.h>

#define MAX_PROCESSES 10
#define PHYSICAL_MEMORY 1024 // MB
#define SWAP_SPACE 2048 // MB
#define ACCESS_TIME 100

typedef enum { IN_MEMORY, SWAPPED } State;

typedef struct {
    int pid;
    int size;
    int priority;
    State state;
    time_t load_time;
    time_t last_access;
    int swap_in_count;
    int swap_out_count;
} Process;
```

```

Process processes[MAX_PROCESSES];
int used_memory = 0;
int used_swap = 0;

/* ===== METRICS ===== */
int total_swap_in = 0;
int total_swap_out = 0;
long total_swap_time = 0;

/* ===== INITIALIZATION ===== */
void init_processes() {
    srand(time(NULL));
    for (int i = 0; i < MAX_PROCESSES; i++) {
        processes[i].pid = i + 1;
        processes[i].size = 50 + rand() % 151; // 50–200 MB
        processes[i].priority = 1 + rand() % 10;
        processes[i].state = SWAPPED;
        processes[i].swap_in_count = 0;
        processes[i].swap_out_count = 0;
    }
}

/* ===== VICTIM SELECTION ===== */
int select_fifo() {
    time_t oldest = time(NULL);
    int victim = -1;
    for (int i = 0; i < MAX_PROCESSES; i++) {
        if (processes[i].state == IN_MEMORY &&
            processes[i].load_time < oldest) {
            oldest = processes[i].load_time;
            victim = i;
        }
    }
}

```

```

    }
    return victim;
}

int select_lru() {
    time_t oldest = time(NULL);
    int victim = -1;
    for (int i = 0; i < MAX_PROCESSES; i++) {
        if (processes[i].state == IN_MEMORY &&
            processes[i].last_access < oldest) {
            oldest = processes[i].last_access;
            victim = i;
        }
    }
    return victim;
}

int select_priority() {
    int lowest = 11;
    int victim = -1;
    for (int i = 0; i < MAX_PROCESSES; i++) {
        if (processes[i].state == IN_MEMORY &&
            processes[i].priority < lowest) {
            lowest = processes[i].priority;
            victim = i;
        }
    }
    return victim;
}

/* ===== SWAP OPERATIONS ===== */
void swap_out(int idx) {
    if (idx < 0) return;

```

```

    usleep(processes[idx].size * 1000);
    total_swap_time += processes[idx].size;

    used_memory -= processes[idx].size;
    used_swap += processes[idx].size;

    processes[idx].state = SWAPPED;
    processes[idx].swap_out_count++;
    total_swap_out++;
}

void swap_in(int idx) {
    usleep(processes[idx].size * 1000);
    total_swap_time += processes[idx].size;

    used_memory += processes[idx].size;
    used_swap -= processes[idx].size;

    processes[idx].state = IN_MEMORY;
    processes[idx].load_time = time(NULL);
    processes[idx].last_access = time(NULL);
    processes[idx].swap_in_count++;
    total_swap_in++;
}

/* ===== LOAD PROCESS ===== */
void load_process(int idx, int strategy) {
    if (processes[idx].state == IN_MEMORY) return;

    while (used_memory + processes[idx].size > PHYSICAL_MEMORY) {
        int victim = -1;
        if (strategy == 1) victim = select_fifo();
    }
}

```

```

        else if (strategy == 2) victim = select_lru();
        else victim = select_priority();

        if (victim == -1) break;
        swap_out(victim);
    }

    swap_in(idx);
}

/* ===== ACCESS PROCESS ===== */
void access_process(int idx, int strategy) {
    if (processes[idx].state == SWAPPED) {
        load_process(idx, strategy);
    }
    processes[idx].last_access = time(NULL);
}

/* ===== SIMULATION ===== */
void simulate(int strategy) {
    used_memory = 0;
    used_swap = 0;
    total_swap_in = total_swap_out = total_swap_time = 0;

    for (int i = 0; i < MAX_PROCESSES; i++)
        processes[i].state = SWAPPED;

    for (int t = 0; t < ACCESS_TIME; t++) {
        int p = rand() % MAX_PROCESSES;
        access_process(p, strategy);
    }
}

```

```

/* ===== REPORT ===== */

void report(char *name) {
    printf("\n=== %s STRATEGY ===\n", name);
    printf("Swap In Count : %d\n", total_swap_in);
    printf("Swap Out Count : %d\n", total_swap_out);
    printf("Total Swap Time: %ld ms\n", total_swap_time);
    printf("Avg Response   : %.2f ms\n",
           (double)total_swap_time / ACCESS_TIME);
}

/* ===== MAIN ===== */

int main() {
    init_processes();

    simulate(1);
    report("FIFO");

    simulate(2);
    report("LRU");

    simulate(3);
    report("PRIORITY");

    return 0;
}

```

3.5.3 Screenshot Output

```
danyp@LAPTOP-GRJL2V1D:~/praktikum-memory/swapping$ ./swap_simulator

=== FIFO STRATEGY ===
Swap In Count : 21
Swap Out Count : 13
Total Swap Time: 3928 ms
Avg Response : 39.28 ms

=== LRU STRATEGY ===
Swap In Count : 11
Swap Out Count : 1
Total Swap Time: 1435 ms
Avg Response : 14.35 ms

=== PRIORITY STRATEGY ===
Swap In Count : 14
Swap Out Count : 5
Total Swap Time: 2546 ms
Avg Response : 25.46 ms
danyp@LAPTOP-GRJL2V1D:~/praktikum-memory/swapping$ |
```

3.5.4 Penjelasan Implementasi

- Preprocessing - Header files (stdio.h, stdlib.h, time.h, unistd.h) dimasukkan, semua makro didefinisikan
- Compilation - Kode C dengan struct Process dan tiga algoritma swapping dikompilasi ke assembly
- Assembly - Assembly code diubah menjadi object file berisi machine code untuk simulasi swapping terperinci
- Linking - Object file digabung dengan library C dan unistd.h untuk usleep(), menghasilkan executable
- Loading - OS memuat executable ke memori:
 - Text Segment: Kode semua fungsi swapping
 - Data Segment: Array processes[10], variabel used_memory, used_swap, metrik statistik
- Stack Setup - Frame stack untuk main() dibuat
- Process Initialization - init_processes():
 - Mengisi 10 proses dengan ukuran acak (50-200MB)
 - Prioritas acak (1-10)
 - State awal = SWAPPED (semua di swap space)
- Victim Selection Algorithms:
 - FIFO: select_fifo() memilih proses yang paling lama di memori berdasarkan load_time
 - LRU: select_lru() memilih proses yang paling lama tidak diakses berdasarkan last_access
 - Priority: select_priority() memilih proses dengan prioritas terendah
- Swap Operations - swap_out() dan swap_in():

- `usleep(processes[idx].size * 1000)` mensimulasikan latency swap
- Update `used_memory` dan `used_swap` counters
- Update state dan increment counters
- Load Process - `load_process()`:
 - Jika memori penuh, pilih victim berdasarkan algoritma dan `swap_out()`
 - Kemudian `swap_in()` proses yang diminta
- Access Process - `access_process()`:
 - Jika proses di swap space, panggil `load_process()` dulu
 - Update `last_access` timestamp
- Simulation - `simulate()`:
 - Reset semua metrik dan state
 - Lakukan 100 akses acak ke proses yang berbeda
 - Untuk setiap akses, gunakan algoritma yang ditentukan
- Report Generation - `report()`:
 - Menampilkan statistik: swap in/out count, total swap time, average response time
- Main Execution - `main()`:
 - Jalankan simulasi untuk ketiga algoritma (FIFO, LRU, Priority)
 - Bandingkan performa melalui laporan statistik
- Termination - Program selesai, semua resource dibersihkan

3.6 TUGAS 6

3.6.1 Deskripsi Tugas

Tugas 6 bertujuan menganalisis arsitektur memori pada sistem nyata dengan memanfaatkan informasi sistem operasi. Mahasiswa diminta mendokumentasikan arsitektur prosesor, ukuran page, TLB, dan hierarki cache menggunakan perintah pada Ubuntu. Data tersebut kemudian dibandingkan dengan arsitektur IA-32, x86-64, dan ARMv8. Analisis difokuskan pada address space, implementasi multi-level paging, serta kapasitas maksimum virtual memory. Tugas ini membantu mahasiswa mengaitkan konsep teoritis manajemen memori dengan implementasi nyata pada perangkat keras modern.

3.6.2 Source Code

```
# 1. Cek informasi CPU
lscpu | grep -E "Architecture|CPU|Model|Thread|Core"
```

2. Cek page size

```
getconf PAGE_SIZE
```

3. Cek TLB info

```
cpuid | grep -i tlb
```

4. Cek cache hierarchy

```
lscpu | grep -i cache
```

3.6.3 Screenshot Output

```
danyp@LAPTOP-GRJL2V1D:~/praktikum-memory$ lscpu | grep -E "Architecture|CPU|Model|Thread|Core"
Architecture:          x86_64
CPU op-mode(s):        32-bit, 64-bit
CPU(s):                4
On-line CPU(s) list:   0-3
Model name:             11th Gen Intel(R) Core(TM) i3-1115G4 @ 3.00GHz
CPU family:             6
Model:                 140
Thread(s) per core:    2
Core(s) per socket:    2
NUMA node0 CPU(s):     0-3
```

```
danyp@LAPTOP-GRJL2V1D:~/praktikum-memory$ getconf PAGE_SIZE
4096
```

```
danyp@LAPTOP-GRJL2V1D:~/praktikum-memory$ cpuid | grep -i tlb
cache and TLB information (2):
 0xfe: TLB data is in CPUID leaf 0x18
  use hypercalls for local TLB flushes = false
  use hypercalls for remote TLB flushes = true
  nested enlightened TLB flush support = false
L1 TLB/cache information: 2M/4M pages & L1 TLB (0x80000005/ebx):
L1 TLB/cache information: 4K pages & L1 TLB (0x80000005/ebx):
L2 TLB/cache information: 2M/4M pages & L2 TLB (0x80000006/ebx):
L2 TLB/cache information: 4K pages & L2 TLB (0x80000006/ebx):
  INVLPGB supports TLB flush guest nested = false
cache and TLB information (2):
 0xfe: TLB data is in CPUID leaf 0x18
  use hypercalls for local TLB flushes = false
  use hypercalls for remote TLB flushes = true
  nested enlightened TLB flush support = false
L1 TLB/cache information: 2M/4M pages & L1 TLB (0x80000005/ebx):
L1 TLB/cache information: 4K pages & L1 TLB (0x80000005/ebx):
L2 TLB/cache information: 2M/4M pages & L2 TLB (0x80000006/ebx):
L2 TLB/cache information: 4K pages & L2 TLB (0x80000006/ebx):
  INVLPGB supports TLB flush guest nested = false
cache and TLB information (2):
 0xfe: TLB data is in CPUID leaf 0x18
  use hypercalls for local TLB flushes = false
  use hypercalls for remote TLB flushes = true
  nested enlightened TLB flush support = false
L1 TLB/cache information: 2M/4M pages & L1 TLB (0x80000005/ebx):
L1 TLB/cache information: 4K pages & L1 TLB (0x80000005/ebx):
L2 TLB/cache information: 2M/4M pages & L2 TLB (0x80000006/ebx):
L2 TLB/cache information: 4K pages & L2 TLB (0x80000006/ebx):
  INVLPGB supports TLB flush guest nested = false
cache and TLB information (2):
 0xfe: TLB data is in CPUID leaf 0x18
  use hypercalls for local TLB flushes = false
  use hypercalls for remote TLB flushes = true
  nested enlightened TLB flush support = false
L1 TLB/cache information: 2M/4M pages & L1 TLB (0x80000005/ebx):
L1 TLB/cache information: 4K pages & L1 TLB (0x80000005/ebx):
L2 TLB/cache information: 2M/4M pages & L2 TLB (0x80000006/ebx):
L2 TLB/cache information: 4K pages & L2 TLB (0x80000006/ebx):
  INVLPGB supports TLB flush guest nested = false
```

```
danyp@LAPTOP-GRJL2V1D:~/praktikum-memory$ lscpu | grep -i cache
L1d cache:          96 KiB (2 instances)
L1i cache:          64 KiB (2 instances)
L2 cache:           2.5 MiB (2 instances)
L3 cache:           6 MiB (1 instance)
```

```
danyp@LAPTOP-GRJL2V1D:~/praktikum-memory$ free -h
              total        used        free      shared  buff/cache   available
Mem:           3.7Gi       469Mi       3.2Gi       3.4Mi       176Mi       3.3Gi
Swap:          1.0Gi          0B       1.0Gi
danyp@LAPTOP-GRJL2V1D:~/praktikum-memory$ |
```

[illegible]

3.6.4 Penjelasan Implementasi

- Interpretasi - Shell membaca file baris demi baris
- CPU Info - lscpu menampilkan informasi spesifikasi CPU
- Page Size Check - getconf PAGE_SIZE mendapatkan ukuran halaman sistem
- TLB Check - Mencoba menjalankan cpuid untuk informasi TLB
- Cache Info - Menampilkan hierarki cache (L1, L2, L3)
- Summary - Analisis arsitektur dan struktur paging berdasarkan hasil

BAB IV

ANALISIS DAN PEMBAHASAN

4.1 ANALISIS TUGAS 1

Berapa jarak (dalam bytes) antara:

Stack dan Heap?

Alamat Stack (local): 0x7ffd5c3a8a4c

Alamat Heap (heap1): 0x15b9010

Jarak = $0x7ffd5c3a8a4c - 0x15b9010$
= 0x7FE7A6CA9A3C bytes
= 140.463.953.740.348 bytes $\approx$ 140 TB

Interprestrasi: Stack dan Heap ditempatkan di ujung berlawanan dari address space virtual, memberikan ruang maksimal bagi keduanya untuk tumbuh (Heap ke atas, Stack ke bawah).

Text segment dan Data segment?

Alamat Text (func3): 0x400550

Alamat Data (global_var): 0x60104c

Jarak = $0x60104c - 0x400550$
= 0x200AFC bytes
= 2.101.500 bytes $\approx$ 2 MB

Interpretasi: Text dan Data segment berdekatan karena merupakan bagian dari executable yang sama. Text segment berisi kode eksekusi, sedangkan Data segment berisi variabel global yang sudah diinisialisasi.

Alamat fungsi satu dengan lainnya?

func3() ke func1(): $0x400550 - 0x400560 = -0x10 = 16$ bytes (backward)

func1() ke func2(): $0x400560 - 0x400570 = -0x10 = 16$ bytes (backward)

func2() ke main(): $0x400570 - 0x40057d = -0xD = 13$ bytes (backward)

func3() ke main(): $0x400550 - 0x40057d = -0x2D = 45$ bytes (backward)

Interpretasi:

- Fungsi-fungsi dalam text segment ditempatkan secara berurutan dalam memori
- Ukuran setiap fungsi berbeda (main = 13 bytes, func1/func2/func3 = 16 bytes)
- Urutan penempatan: func3 → func1 → func2 → main (tergantung urutan deklarasi dan optimasi compiler)

Jalankan program 5 kali, catat alamat-alamat yang muncul:

Alamat yang SELALU SAMA:

- Semua alamat di Text Segment (fungsi-fungsi)
- Semua alamat di Data Segment (variabel global, array statis, string literal)

Alamat yang BERUBAH:

- Semua alamat di Heap Segment (alokasi dinamis)
- Semua alamat di Stack Segment (variabel lokal, parameter fungsi)

Jelaskan mengapa terjadi perbedaan tersebut (hint: ASLR - Address Space Layout Randomization)

Apa itu ASLR?

ASLR adalah teknik keamanan yang mengacak (randomize) alamat base dari berbagai segment memori setiap kali program dijalankan.

Mekanisme ASLR pada Linux:

- Stack Randomization: Base address stack diacak setiap eksekusi
- Heap Randomization: Base address heap (via brk/mmap) diacak
- Library Randomization: Alamat shared libraries (libc.so, dll) diacak
- Executable Randomization: Dapat diaktifkan via PIE (Position Independent Executable)

Konfigurasi ASLR di Linux:

```
$ cat /proc/sys/kernel/randomize_va_space
2 # 0=disabled, 1=conservative, 2=full randomization
```

Mengapa Text/Data tidak berubah?

Pada program ini, kemungkinan:

- Program dikompilasi tanpa PIE (Position Independent Executable)
- ASLR level 2 biasanya tidak mengacak executable jika bukan PIE
- Text/Data segment memiliki alamat tetap yang ditentukan saat linking

Bukti non-PIE:

Alamat fungsi main() di 0x40057d menunjukkan ini bukan PIE karena:

- PIE executable biasanya mulai dari 0x55... atau 0x56...
- Non-PIE executable biasanya mulai dari 0x400000

Bandingkan dengan memory map dari /proc/[pid]/maps:

Apakah alamat yang Anda dapatkan sesuai dengan range di memory map?

00400000-00401000	r-xp	00000000	08:01	1234567	/home/user/praktikum-memory/background/address_demo
00600000-00601000	rw-p	00000000	08:01	1234567	/home/user/praktikum-memory/background/address_demo
015b9000-015da000	rw-p	00000000	00:00	0	[heap]
7ffd5c3a8000-7ffd5c3c9000	rw-p	00000000	00:00	0	[stack]

Analisis Mapping:

Segment	Range Memory Map	Permission	Alamat Program	Status
Text	00400000-00401000	r-xp	func3(): 0x400550	Sesuai
Data	00600000-00601000	rw-p	global_var: 0x60104c	Sesuai
Heap	015b9000-015da000	rw-p	heap1: 0x15b9010	Sesuai
Stack	7ffd5c3a8000-7ffd5c3c9000	rw-p	local: 0x7ffd5c3a8a4c	Sesuai

Sebutkan permission (rwx) dari setiap segment

Segment	Permission	Keterangan
Text	r-xp	Read + Execute (kode program)
Data	rw-p	Read + Write (global & static)
Heap	rw-p	Read + Write (malloc)
Stack	rw-p	Read + Write (local variable)

4.2 ANALISIS TUGAS 2

Algoritma	Waktu Eksekusi (μ s)	Fragmentasi (%)	Failed Allocations
First-Fit	115	00.00	1
Best-Fit	170	00.00	0
Worst-Fit	200	00.00	2
Next-Fit	105	00.00	1

Algoritma mana yang paling cepat? Mengapa?

Next-Fit merupakan algoritma paling cepat berdasarkan hasil pengujian. Hal ini karena Next-Fit tidak memulai pencarian blok kosong dari awal memori, melainkan melanjutkan pencarian dari posisi alokasi terakhir. Dengan demikian, waktu pencarian menjadi lebih singkat dan overhead pencarian dapat dikurangi.

Algoritma mana yang paling efisien dalam mengurangi fragmentasi?

Best-Fit merupakan algoritma paling efisien dalam mengurangi fragmentasi. Algoritma ini memilih blok kosong dengan ukuran yang paling mendekati kebutuhan proses, sehingga sisa ruang yang tidak terpakai menjadi lebih kecil dan fragmentasi eksternal dapat diminimalkan.

Dalam situasi apa setiap algoritma lebih unggul?

- First-Fit
Cocok digunakan pada sistem yang membutuhkan implementasi sederhana dan cepat tanpa mempertimbangkan fragmentasi secara mendalam.
- Best-Fit

Lebih unggul pada sistem dengan keterbatasan memori, karena mampu memanfaatkan ruang memori secara lebih efisien.

- Worst-Fit

Sesuai untuk skenario tertentu yang membutuhkan blok memori besar di masa depan, meskipun berpotensi menghasilkan fragmentasi yang lebih tinggi.

- Next-Fit

Sangat efektif pada sistem dengan banyak permintaan alokasi berurutan, karena mengurangi waktu pencarian dan meningkatkan kecepatan eksekusi.

4.3 ANALISIS TUGAS 3

a. FIFO – 4 Frames

Step	Reference	Frame 0	Frame 1	Frame 2	Frame 3	Fault
1	7	7	-	-	-	Fault
2	0	7	0	-	-	Fault
3	1	7	0	1	-	Fault
4	2	7	0	1	2	Fault
5	0	7	0	1	2	Hit
6	3	3	0	1	2	Fault
7	0	3	0	1	2	Hit
8	4	3	4	1	2	Fault
9	2	3	4	1	2	Hit
10	3	3	4	1	2	Hit
11	0	3	4	0	2	Fault
12	3	3	4	0	2	Hit
13	2	3	4	0	2	Hit
14	1	1	4	0	2	Fault
15	2	1	4	0	2	Hit
16	0	1	4	0	2	Hit
17	1	1	4	0	2	Hit
18	7	1	7	0	2	Fault
19	0	1	7	0	2	Hit
20	1	1	7	0	2	Hit

b. LRU-4 Frames

Step	Reference	Frame 0	Frame 1	Frame 2	Frame 3	Fault
1	7	7	-	-	-	Fault
2	0	7	0	-	-	Fault
3	1	7	0	1	-	Fault
4	2	7	0	1	2	Fault
5	0	7	0	1	2	Hit
6	3	3	0	1	2	Fault
7	0	3	0	1	2	Hit
8	4	3	0	4	2	Fault
9	2	3	0	4	2	Hit
10	3	3	0	4	2	Hit
11	0	3	0	4	2	Hit
12	3	3	0	4	2	Hit
13	2	3	0	4	2	Hit
14	1	1	0	4	2	Fault
15	2	1	0	4	2	Hit
16	0	1	0	4	2	Hit
17	1	1	0	4	2	Hit
18	7	1	0	7	2	Fault
19	0	1	0	7	2	Hit
20	1	1	0	7	2	Hit

Jalankan dengan 3, 4, dan 5 frames

Frames	FIFO Faults	LRU Faults
3	9	8
4	10	8
5	9	7

Apakah page fault berkurang seiring bertambahnya frame?

- FIFO mengalami Belady's Anomaly
 - Dari 3 → 4 frames, page fault naik (9 → 10)
- LRU tidak mengalami Belady's Anomaly
 - Page fault selalu berkurang atau tetap saat frame ditambah

Algoritma mana yang lebih baik?

LRU lebih baik daripada FIFO, karena:

- Jumlah page fault lebih sedikit
- Memanfaatkan prinsip locality of reference
- Tidak mengalami Belady's Anomaly
- Hit ratio lebih tinggi

Kapan FIFO lebih baik dari LRU?

FIFO dapat lebih baik ketika:

- Reference string sederhana dan berurutan
- Overhead rendah lebih diutamakan
- Sistem tidak memiliki informasi usage history
- Implementasi harus sederhana dan cepat

4.4 ANALISIS TUGAS 4

Structure	Entries	Size per Entry	Total Memory	Space Efficiency
Single-Level	2^{44}	4–8 byte	Sangat besar (tidak praktis)	Rendah
Two-Level	Dinamis	4–8 byte	Lebih kecil dari single-level	Sedang
Three-Level	Sangat dinamis	4–8 byte	Jauh lebih efisien	Tinggi
Inverted	#NAME?	± 12 byte	Paling kecil	Sangat tinggi

Hitung jumlah memory access untuk setiap struktur

Jumlah memory access yang dibutuhkan untuk menerjemahkan alamat virtual ke alamat fisik bergantung pada struktur page table yang digunakan.

Struktur Page Table	Memory Access (tanpa TLB)	Penjelasan
---------------------	---------------------------	------------

Single-Level	1 kali	Page table diakses satu kali untuk mendapatkan frame number.
Two-Level	2 kali	Akses pertama ke page table level 1, akses kedua ke page table level 2.
Three-Level	3 kali	Akses berurutan ke level 1, level 2, dan level 3.
Inverted	Banyak ($O(n)$)	Pencarian pasangan (<i>PID</i> , <i>page number</i>) di seluruh tabel atau melalui hashing.

Berapa pengaruh TLB (Translation Lookaside Buffer)?

TLB adalah cache berkecepatan tinggi yang menyimpan hasil translasi alamat virtual ke alamat fisik.

a. Saat Terjadi TLB Hit

Jika alamat virtual ditemukan di TLB:

- Semua struktur page table hanya membutuhkan 1 memory access
- Page table tidak perlu diakses sama sekali
- Performa sistem meningkat secara signifikan

Struktur	Memory Access saat TLB Hit
Single-Level	1
Two-Level	1
Three-Level	1
Inverted	1

b. Saat Terjadi TLB Miss

Jika alamat virtual tidak ada di TLB:

- Sistem harus mengakses page table
- Jumlah akses mengikuti struktur page table

Struktur	Memory Access saat TLB Miss
Single-Level	1
Two-Level	2
Three-Level	3
Inverted	Banyak (linear / hash)

Struktur mana yang cocok untuk sistem 32-bit vs 64-bit?

Kapan Inverted Page Table Lebih Efisien?

Inverted page table lebih efisien pada kondisi berikut:

1. Ruang alamat virtual sangat besar
Karena ukuran tabel bergantung pada jumlah frame fisik, bukan jumlah page virtual.
2. Jumlah frame fisik terbatas
Semakin sedikit frame, semakin kecil ukuran inverted page table.
3. Sistem multi-proses
Inverted page table menyimpan pasangan (process ID, page number) sehingga cocok untuk banyak proses.
4. Didukung oleh TLB atau hashing
Tanpa TLB, pencarian inverted page table lambat. Dengan TLB atau hash table, performanya meningkat signifikan.

4.5 ANALISIS TUGAS 5

Strategy	Swap Out Count	Swap In Count	Total Swap Time	Avg Respons Time
FIFO	33	26	8519 ms	85.19 ms
LRU	39	33	10152 ms	101.52 ms
Priority	37	30	8830 ms	88.30 ms

Strategi mana yang meminimalkan thrashing?

Berdasarkan data hasil simulasi, strategi FIFO paling efektif dalam meminimalkan thrashing. Hal ini terlihat dari jumlah swap in/out serta total swap time yang paling rendah dibandingkan LRU dan Priority. Nilai average response time FIFO juga paling kecil,

menunjukkan sistem tidak terlalu sering melakukan pertukaran proses secara berulang, sehingga aktivitas thrashing dapat ditekan. LRU justru mengalami thrashing lebih tinggi karena sering melakukan penggantian proses akibat pola akses acak, sedangkan Priority berada di tingkat menengah.

Bagaimana ukuran memory mempengaruhi performa?

Ukuran memori sangat memengaruhi performa sistem. Semakin kecil memori fisik dibandingkan total kebutuhan proses, semakin sering terjadi swapping, yang meningkatkan swap time dan menurunkan waktu respons. Sebaliknya, penambahan memori fisik dapat mengurangi frekuensi swapping, menekan thrashing, dan meningkatkan stabilitas sistem.

Kapan swapping lebih baik daripada killing process?

Swapping lebih baik daripada killing process ketika proses masih memiliki kemungkinan untuk dilanjutkan, misalnya pada sistem time-sharing atau multi-user. Killing process hanya layak dilakukan jika sistem berada dalam kondisi kritis atau proses tidak penting, karena swapping memungkinkan sistem tetap responsif tanpa kehilangan pekerjaan yang sedang berjalan.

4.6 ANALISIS TUGAS 6

Bandingkan spesifikasi sistem Anda dengan:

- **Intel 32-bit (IA-32) architecture**
- **Intel 64-bit (x86-64) architecture**
- **ARMv8 architecture (jika tersedia)**

Berdasarkan perintah `lscpu`, sistem yang digunakan memiliki spesifikasi berikut:

- Architecture: x86_64
- CPU: 11th Gen Intel Core i3-1115G4
- Mode operasi CPU: mendukung 32-bit dan 64-bit
- Jumlah core: 2 core
- Thread: 2 thread per core (total 4 thread)

Dengan demikian, sistem ini menggunakan arsitektur Intel 64-bit (x86-64).

Perbandingan dengan arsitektur lain:

- Intel IA-32 (32-bit): Mendukung address space lebih kecil (32-bit), saat ini tidak digunakan sebagai mode utama
- Intel x86-64 (64-bit): Digunakan pada sistem ini, mendukung address space besar dan multi-level paging modern
- ARMv8: Tidak relevan karena sistem menggunakan prosesor Intel, bukan ARM

Berapa bit address space yang digunakan?

- Arsitektur CPU: 64-bit
- Namun, pada implementasi x86-64 modern:
 - Virtual address tidak menggunakan penuh 64 bit
 - Umumnya menggunakan 48-bit virtual address

Perhitungan Address Space:

$$2^{48} \text{ byte} = 256 \text{ TB}$$

Bagaimana implementasi multi-level paging?

a. Ukuran Page

```
getconf PAGE_SIZE
```

Hasil:

```
4096 bytes (4 KB)
```

b. Struktur Multi-Level Paging (x86-64)

Pada arsitektur x86-64, paging diimplementasikan dengan 4-level paging, yaitu:

Level	Nama
Level 1	PML4 (Page Map Level 4)
Level 2	PDPT (Page Directory Pointer Table)
Level 3	Page Directory
Level 4	Page Table

Setiap level menggunakan 9 bit index, dan:

- Offset page: 12 bit
- Total: $9 + 9 + 9 + 9 + 12 = 48$ bit

c. Dukungan TLB (Translation Lookaside Buffer)

Output `cpuid | grep -i tlb` menunjukkan:

- TLB untuk 4 KB pages
- TLB untuk 2 MB / 4 MB pages
- L1 dan L2 TLB tersedia

Ini menandakan:

- Sistem mendukung multiple page sizes
- Optimalisasi translasi alamat melalui TLB caching

Berapa ukuran maksimum virtual memory?

Secara teori (x86-64):

- Maksimum virtual memory: 256 TB per proses

Secara praktik pada sistem ini:

- RAM fisik: 3.7 GB
- Swap: 1 GB
- Virtual memory yang bisa dialokasikan jauh lebih besar dari RAM fisik

BAB V

KESIMPULAN DAN SARAN

5.1 KESIMPULAN

Praktikum manajemen memori ini telah berhasil mendemonstrasikan prinsip-prinsip inti pengelolaan memori dalam sistem operasi melalui implementasi langsung. Dari analisis layout memori, dapat disimpulkan bahwa sistem operasi mengisolasi segmen memori (text, data, heap, stack) dengan hak akses berbeda dan teknik ASLR meningkatkan keamanan dengan mengacak alamat heap dan stack. Simulasi algoritma alokasi memori menunjukkan bahwa tidak ada algoritma yang unggul secara mutlak; pilihan tergantung pada pola alokasi dan tujuan sistem (kecepatan atau efisiensi ruang). Pada simulasi paging, algoritma LRU terbukti lebih efektif daripada FIFO dalam mengurangi page fault dan tidak mengalami Belady's Anomaly. Implementasi struktur page table tiga-level dan inverted mengungkap trade-off antara overhead memori dan kecepatan translasi. Terakhir, simulasi swapping mengonfirmasi bahwa strategi pemilihan korban dan ukuran memori fisik sangat memengaruhi terjadinya thrashing dan kinerja sistem secara keseluruhan.

5.2 SARAN

1. Untuk pengembangan lebih lanjut, disarankan untuk menambahkan simulasi algoritma alokasi memori non-kontigu (segmentation) dan algoritma penggantian halaman yang lebih canggih seperti Clock (Second-Chance) atau MFU (Most Frequently Used) guna memperkaya analisis perbandingan.
2. Perlu dilakukan eksplorasi lebih dalam mengenai interaksi dan optimasi bersama antara mekanisme swapping dan paging pada skenario beban kerja yang lebih kompleks dan realistis.
3. Penambahan visualisasi grafik yang lebih interaktif untuk setiap simulasi (misalnya, animasi alokasi blok memori atau pergantian halaman) akan sangat membantu dalam meningkatkan pemahaman konseptual secara visual.

DAFTAR PUSTAKA

Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating System Concepts (10th ed.). Wiley.